IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

*Digital Object Identifier 10.1109/ACCESS.2017.DOI*

# Process Injection using Return-Oriented Programming

**BRAMWELL BRIZENDINE[1], SHIVA SHASHANK KUSUMA[1], BHASKAR P. RIMAL[2] (Senior Member, IEEE)**

[1]Department of Computer Science, University of Alabama in Huntsville, Huntsville, AL 35899 USA (email: bjb0047@uah.edu; sk0190@uah.edu)
[2]Department of Computer Science, University of Idaho, Moscow, ID 83844-1010, USA (e-mail: bhaskar.rimal@ieee.org)

Corresponding author: Bramwell Brizendine(e-mail: bjb0047@uah.edu).

This work was supported in part by Dr. Brizendine's Startup Fund with the University of Alabama in Huntsville, Huntsville, AL 35899 USA

**ABSTRACT** Return-oriented programming (ROP) is a code-reuse attack that uses borrowed chunks of executable code for arbitrary computation. On Windows, ROP is often used solely to bypass Data Execution Prevention, rather than realizing its full potential; indeed, the bulk of advanced, malicious functionality is typically invoked through shellcode. This paper demonstrates an approach to advanced process injection using only ROP that works without shellcode or higher-level code, providing significantly more advanced functionality than is typically achieved with ROP. We show how to generalize a complex exploit chain that invokes advanced malicious functionality consisting of multiple function calls that are made using only ROP. We generalize this approach by creating a library of nearly 150 parameter-loading patterns, thereby making process injection as a complete ROP technique portable across multiple, dissimilar binaries. Previously, only a few APIs were documented with parameter-loading patterns, making advanced ROP techniques on Windows less straightforward. Experimental validation on several Windows builds confirms our approach is reliable on the modern Windows operating system. This work advances the state-of-the-art in offensive security, offering a foundation for future research, both defensively and in software exploitation.

**INDEX TERMS** Return-Oriented Programming (ROP), Reverse Engineering, Malware Defense, Data Execution Prevention (DEP)

## I. INTRODUCTION

On Windows, Return-Oriented Programming (ROP) is most frequently used to bypass mitigations such as Data Execution Prevention (DEP), making it possible to execute shellcode and achieve arbitrary code execution [1], [2]. However, our extensive analysis of ROP chains used in the wild and publicly shared revealed a significant gap between the theoretical capabilities of ROP and real-world usage. The most complex chains documented in practice typically invoke no more than three WinAPIs—such as `LoadLibraryA`, `GetProcAddress`, and `System`—to perform straightforward tasks like adding a user [3]. Though a close examination of publicly available ROP exploits shows a vast majority use only a single WinAPI, often to bypass DEP. This approach falls far short of ROP's potential sophistication, showing a wide gap between what is possible and what has been implemented in the wild.

Shellcode can dynamically resolve the runtime address of WinAPI addresses, allowing for highly complex, nuanced WinAPI usage. Windows programming often requires generating different handles via various WinAPIs, returned via `EAX`, out parameters, or even through output parameters. On Windows, these challenging tasks often would be completed in shellcode, not directly via ROP. Several challenges have contributed to this status quo. First, chaining multiple WinAPIs via ROP is inherently complex, since most rely on the PUSHAD instruction to load parameters into registers [4]. ROP patterns act as blueprints for register assignment, as once PUSHAD executes, values land on the stack in a predictable order. Typically on Windows, most ROP chains will be designed so that they can have the stack set up with the parameters and return value, allowing for the WinAPI to be called in one of a variety of ways. `VirtualAlloc` and `VirtualProtect` are the most well-known examples of using patterns with PUSHAD, each with two examples. Thus, often the goal is to load values into registers and then execute the `PUSHAD` instruction [4]. Using `VirtualAlloc` or `VirtualProtect` can bypass DEP, allowing for immedi-

ate shellcode execution [5]. Defining effective ROP patterns has rarely been attempted for most WinAPIs, leaving even advanced users to work out these details manually. Creating such a pattern itself can be complex and non-trivial. For most APIs, the patterns are undefined, and while a user could figure them out, no clear methodology is documented, nor is it necessarily straightforward.

In addition, chaining multiple WinAPIs together via ROP to perform advanced functionality, akin to shellcode, can require substantial effort and deep WinAPI knowledge. Bypassing DEP [1], [2] to execute shellcode is the norm on Windows, as it is less labor-intensive than a ROP-only approach and avoids the myriad problems of advanced ROP usage.

A further difficulty lies in performing process injection entirely via ROP. It requires string comparisons to identify the correct target process, a required step for capturing the process identifier (PID). Because string-comparison gadgets are often scarce in ROP, this step becomes especially daunting. Without the ability for numerous string comparisons, an ROP-only process injection is infeasible on Windows.

We attempt to bridge these gaps from what is theoretically possible to what has been done and is practical, by showing that advanced functionality can be achieved solely with ROP. This also avoids the need for shellcode. To illustrate ROP's broader potential, we chose process injection as an in-depth case study, given its complexity and many variations [6], [7]. Process injection may use a dozen or more WinAPIs, requiring memory allocation, privilege escalation, enumeration of running processes, code injection, and thread creation. By handling this entire WinAPI sequence with nothing more than ROP gadgets, we show that advanced functionality is possible and that similarly complex tasks become feasible. Process injection is ideal because MITRE [8] has documented its many variations, though it is only minimally covered in academic literature. Historically, ROP has been used sparingly within the broader workflow of process injection, with most process injection techniques written almost entirely in higher-level programming languages. Direct usage of ROP in process injection has been limited to just a few short sequences.

One difficulty in a fully ROP-driven process injection is the lack of gadgets for string comparisons. In programming, checking each character of a process name is simple, but in an ROP it requires specialized Assembly instructions that may not exist as gadgets. While that is possible in some target applications, such gadgets are rare in most binaries. This would preclude an ROP-only process injection since identifying a specific PID is required. Thus, an ROP-only process injection might be impossible or unlikely for many binaries, given the difficulty of finding suitable gadgets to implement string comparisons and conditional logic.

We overcome those limitations by creating nearly 150 reusable ROP patterns. This greatly extends the overall attack surface, allowing for alternative patterns to be substituted when one potential chain appears infeasible. For instance, one pattern might assign a parameter value to `EDI`, while another uses `EBX`. If `EDI` lacks sufficient gadgets, `EBX` might provide other viable possibilities. Multiple patterns for each WinAPI help researchers overcome gadget scarcity. Because most WinAPI patterns are undocumented, offering them represents a major contribution to offensive security. After all, given the non-trivial nature of devising patterns, even creating one pattern could be time-intensive and require advanced knowledge. As part of this research, we documented all available patterns of the WinAPIs used in the variants of process injection we explored.

We introduce a fully ROP-only approach to process injection. Unlike methods that rely primarily on higher-level languages or a hybrid approach combining minimal ROP with other code, ours demonstrates that all injection stages—allocation, privilege escalation, and payload execution—are feasible with ROP alone. We also document complete ROP patterns for each WinAPI used in process injection, significantly expanding the attack surface. These patterns give a strong foundation for implementing ROP chains that extend the practical boundaries of real-world exploitation techniques, such as process injection, using only ROP. Our creation of numerous WinAPI patterns, not previously documented, makes process injection more practical, and it also provides flexibility for adapting to other non-trivial ROP chains. We also evaluate these techniques against modern defenses, including DEP [1], [2] and endpoint detection and response (EDR), showing its portability across Windows builds.

Finally, we extend the conceptual boundaries of ROP on Windows, illustrating how functionality typically performed by shellcode can be achieved purely via ROP. By implementing the functionality traditionally associated with shellcode via ROP chains alone and producing a rich library of patterns, each with all possible variations, we successfully generalize process injection, making our approach easily repeatable on other binaries. With this, we advance the state of the art in ROP and in process injection itself, within the Windows ecosystem. This work will show how as many as 30 WinAPIs can be used in conjunction, a tenfold increase compared to what at present seems to be the upper limit of what an advanced ROP chain might use on Windows. With the creation of these patterns, and by detailing the nuances and practical considerations for reliably chaining these APIs, we provide researchers and practitioners with a foundation for using ROP in red-team efforts. We demonstrate that process injection is no longer limited to injected shellcode and reflective DLLs, providing an alternative with a ROP-only approach. This offers practical advantages that implementing the technique by shellcode cannot match:

- **Reduced Static Signatures and Heuristics:** Typical shellcode uses sequences of machine instructions that antivirus and EDR engines can detect statically or heuristically. Shellcode often uses recognizable decoder stubs or techniques, such as walking the Process Environment Block (PEB), to resolve runtime API addresses, which can be detected via YARA rules. In contrast, our

ROP chains execute from previously loaded modules, leaving fewer recognizable patterns during static or dynamic analysis.

- **Benign Thread Creation:** Process injection implemented via shellcode often spawns or hijacks threads, which can sometimes be detected by scanning for new threads that point to suspicious memory regions. Given that our method uses only ROP to make legitimate WinAPI calls, thread behavior appears more benign. Return addresses lead to previously loaded modules (ROP gadgets), rather than injected shellcode buffers.
- **No Reflective DLL or PE File Dropped:** Our ROP approach does not introduce any new PE file; every instruction executed is from a signed, previously loaded module. Each "instruction" is effectively a gadget address rather than actual code bytes; the ROP chain is a list of 32-bit addresses and constants. As a result, static analysis tools such as VirusTotal or Hybrid Analysis report zero detections, as shown in Section V.
- **No Need to Bypass DEP:** DEP protects against newly injected code. A ROP-only approach sidesteps DEP entirely by not introducing new code to be executed. In contrast, shellcode typically must defeat DEP via methods that are well known and that might be detected.
- **Fewer Observable Events:** By not implementing process injection via shellcode, we reduce multiple observable events that security tools could monitor.

Our approach offers stealth, portability, and extensibility that typical shellcode implementations cannot match. This also makes it clear that defenders must more carefully monitor and evaluate complex ROP behavior that implements malicious functionality.

### A. CONTRIBUTIONS

The key contributions of this paper are summarized as follows:

- We show that a full process injection technique can be done solely through ROP, eliminating reliance on shellcode or using higher-level code (C/C++, C).
- We create a custom `EnumerateProcesses` function, built via ROP by writing the needed opcodes into memory and making them executable, without using shellcode. It solves the long-standing problem of performing both conditional logic and string comparisons via only ROP, a task previously considered extremely difficult, given what often is the scarcity of required gadgets. String comparison otherwise would be nearly impossible using commonly available ROP gadgets.
- We present a comprehensive library of nearly 150 interchangeable, reusable patterns for APIs on Windows for process injection, allowing the technique to be generalized and applicable to a wide array of binaries. Previously, only two or three patterns existed for a handful of WinAPIs (`VirtualAlloc`, `VirtualProtect`). Our expanded library provides multiple ways to set API parameters with only ROP, overcoming gadget scarcity and making complex ROP chains practical across diverse binaries.
- We provide empirical evidence, using results from VirusTotal, HybridAnalysis, Windows Defender, Wazuh EDR, and Elastic EDR, indicating that ROP-only process injection remains undetected by these tools. Our findings suggest that chains reusing legitimate modules and avoiding shellcode signatures can help avoid EDR heuristics.
- We demonstrate the adaptability of patterns, showing stable performance across multiple builds of Windows 10 and 11. This validates that the technique is not tied to any single OS version.
- We formalize a repeatable approach to process injection via ROP, using our reusable patterns and a novel, hybrid method that combines PUSHAD with `MOV` dereference. This approach lets researchers scale process injection to dozens of API calls in one chain, greatly reducing the trial and error needed for multi-API exploits.
- We lay the groundwork for future automated ROP construction, detailing how to build and chain large sets of WinAPIs using ROP. Our library of patterns can be dynamically matched to the gadgets present in a given binary, providing a blueprint for automated tooling that could generate ROP chains for complex tasks.
- Lastly, we illustrate how highly complex, previously impractical tasks–such as requiring multiple handles, structures, and privileged operations–can be invoked on Windows via only ROP, supporting malicious functionality that would otherwise be monumentally difficult.

By documenting these novel contributions, we advance the state-of-the-art in offensive security, providing an actionable roadmap for researchers and practitioners. As with all offensive security efforts, this work provides opportunities to improve defensive security by giving researchers and defenders a more realistic view of what could be done by a dedicated attacker with potentially unlimited resources, without the need to use shellcode or higher-level code.

### B. STRUCTURE OF THE PAPER

The remainder of this paper is organized as follows: Section II discusses the background and related work, covering the evolution of ROP and its usage in process injection. Section III describes the methodology of our approach, focusing on how ROP techniques can be generalized for complex operations, such as process injection. In support of this, we discuss our approach to creating a custom `EnumerateProcesses` function for identifying target processes, how we handle string comparisons using ROP, and the methods we use to construct a library of parameter-loading patterns relevant to process injection. Section IV presents the detailed implementation steps of our proposed approach, illustrating our approach to process injection using ROP. Section V discusses the experimental validation and findings, providing execution metrics and detection results

against EDR. Finally, Section VII concludes the paper and suggests potential future research directions.

## II. BACKGROUND AND RELATED WORK

Code-reuse attacks involve reusing bits of executable code that exist in process memory in an unintended fashion; without the need to introduce new code, existing chunks of code can be repurposed to achieve arbitrary computation by arranging in a particular way. Code-reuse attacks originated with return-to-libc attacks; this involved having programs return to existing functions, typically found in the C library, while also passing needed parameters. This could allow one or more functions to be used in an unintended fashion, often maliciously, as was famously first documented by Solar Designer in 1997 [10].

This prompted mitigations, such as the introduction of ASLR in 2001, to make return-to-libc style attacks harder to implement [11], [12]. Much later in 2004, Microsoft finally introduced DEP, which enforces that code be either readable and writable or executable, but not both, thereby preventing the introduction of shellcode – or position-independent Assembly code – from being introduced directly after a buffer overflow attack and allowing it to execute. The idea of returning arbitrarily to process memory, once execution was captured, then evolved to returning to chunks of executable code, or gadgets, each performing some small task, as introduced by Shacham as ROP in 2007 [13], [14]. As each gadget ends in a ret, each of these short snippets of code could be combined in a ROP chain, to achieve Turing-complete, arbitrary computation [13], [14]. Since that time, ROP has been expanded to all or virtually all architectures, including ARM and x86_64. This also led to tools such as ROPGadget [15], Mona [16], Ropper [17], ROP ROCKET [18], and various others, capable of enumerating ROP gadgets or even constructing ROP chains through automation.

On modern Windows platforms, ROP typically attempts to disable DEP and then pivot to shellcode, often using `VirtualProtect` or `VirtualAlloc`. While such usage is extremely common in real-world exploits, as a close survey of exploits at ExploitDB [19] shows, it barely scratches the surface of ROP's theoretical capabilities on Windows, especially if multiple WinAPIs are chained together for advanced tasks. On ExploitDB, the ROP chain with the most WinAPIs had only three [3]. Yet ROP's complexity and limited documentation of parameter-loading patterns, which can facilitate other usage of WinAPIs in ROP, largely have hindered deeper real-world adoption on Windows beyond mere DEP bypass.

In 2010, Bletsch et al. [20] introduced Jump-Oriented Programming (JOP), which uses indirect jumps and a dispatcher table to achieve control flow, instead of the stack and the RET [21]. JOP was considerably more difficult to use. Subsequently, JOP ROCKET [22], [23] was a tool released that allowed for JOP gadgets to be discovered, and which also created complete JOP chains to bypass DEP with VirtualProtect or VirtualAlloc. In addition, Just-In-Time ROP (JIT-ROP) was introduced by Snow et al., [24] showing how loaded code segments could be scanned and used to dynamically create gadget chains on the fly.

Control-Flow Integrity (CFI) [5], [25] has been introduced to make code-reuse attacks more challenging. CFI efforts led to Microsoft's CFG, introduced in Windows 8.1, but it is imperfect and only applicable to binaries compiled with CFG. In 2015, the NSA introduced a proposal for an improved CFI, which included new Assembly instructions and a shadow stack [26]. A CFI defense released by Intel [27], Control Enforcement Tracking (CET), appeared to borrow heavily from the core concepts of this proposal, although it requires code to be compiled to support it, and the hardware requires a special chip. While CET provides defense against ROP and is a step in the right direction, billions of computers lack the hardware support, and it can be bypassed [28]. In fact, ROP persists and be an ever-relevant means of attack; while there are many mitigations, many can easily be defeated, such as DEP [29]. ASLR [30], [31] presents some challenges, but often it can be overcome with partial memory disclosures.

### A. PROCESS INJECTION

Process injection is an exploitation technique that allows attackers to inject code into the virtual memory of another process, where they can then somehow cause it to be executed. Ideally, this execution from the attacker's standpoint is not detected, and thus it is typically a focal point of their efforts. As such, there are numerous variants of process injection. Process injection often is a goal because it can enable an attacker to migrate from a potentially unstable process to one where there is greater stability and less likely to be detected, as security tools that only monitor the original process might miss the malicious activity being performed in another process [6], [32]. In addition, sometimes process injection results in elevated privileges, as attackers can target a process with an access token that grants privileges to desired resources [33].

Process Injection often involves writing some type of malicious payload into the virtual memory of a running process and then forcing that process to execute the injected payload. Though it varies widely, the writing component sometimes uses `VirtualAllocEx` and `WriteProcessMemory`, or one of various stealthier options, such as mapping sections, shared memory, etc. Execution must then be initiated, and this is often done with methods like creating a remote thread, queuing an APC, or hijacking an existing thread's context.

Typically, there are multiple stages, involving memory allocation, writing, and execution, which causes the payload to be detected [6], [34], [7]. Process injection remains a trendy area of focus with significant offensive security research being performed, to develop novel process injection techniques, by independent researchers or cybersecurity companies [6], [33]. However, there is only very little academic research devoted exclusively to Windows process injection techniques, although there has been some recent work, e.g., [6] and [33]. However, complete, full-process injection code is not often

**TABLE 1.** Comparison of process injection techniques.

| Techniques | Features | Limitations |
|---|---|---|
| DLL Injection | Uses `LoadLibrary` to inject DLL into target | DLL listed in module lists; unsigned DLL can be blocked |
| Process Hollowing | Suspends a process, unmaps it, and inserts malicious code | Unmapping can be complicated; some EDRs detect hollowed sections |
| APC Injection | Queues code with `QueueUserAPC` on a thread in alertable wait | Thread must be alertable; injected code address must be valid under CFG |
| SetWindowsHookEx | Registers a global hook and forces victim to load malicious code | Can be stopped by code-sign checks by Code Integrity Guard (CIG) |
| Remote Thread Injection | Classic approach: `VirtualAllocEx` and `WriteProcessMemory` + `CreateRemoteThread` | APIs often monitored; new thread creation is an IOC |
| Thread Hijacking | Modifies `RIP` of a suspended thread instead of creating a new one | Must restore registers or crash will ensue |
| Reflective DLL Injection | Embeds custom loader inside the DLL to avoid `LoadLibrary` | More complex mapping |
| Stackbombing | Overwrites a victim thread's stack so it eventually returns to malicious code; briefly uses ROP | Delicate stack setup; failing to restore registers causes crash |
| Ghost-writing [7], [9] | Uses repeated context changes with `SetThreadContext` to write data; briefly uses ROP | Complex stack setup; must restore original context carefully |
| Memory Mapping Injection | Uses `NtMapViewOfSection` or shared sections for stealth writes | More advanced; Unable to write bytes previously allocated memory |
| Proposed ROP-only Approach | Fully ROP; can help evade detection and obscure analysis | Complicated setup |

shared, as it may be kept confidential, as distributing them can increase the chance of their being detected in the wild.

Advanced or novel process injection techniques may exploit obscure APIs and unconventional setups, to make detection more challenging. Process injection on Windows can be divided into broad categories, much of which is described in recent literature [6], [7], [35]–[37]. The MITRE ATT&CK framework also provides its own independent categorization of process injection techniques and sub-techniques, although they only document those observed the wild. We can next survey some of these various process injection techniques.

One popular approach is sometimes called classic code injection, where a region of memory is allocated in the target process with `VirtualAllocEx`, and code is written there with `WriteProcessMemory`; a new thread is then spawned with `CreateRemoteThread` or `NtCreateThreadEx`. This technique is simple but can be flagged by EDR [35]–[37].

Classic DLL Injection allows a DLL to be injected into the virtual memory of another process. With this approach, a malicious process obtains a handle to a target process, such as with `OpenProcess`, and then allocates memory remotely, e.g. via `VirtualAllocEx`. It can then open the DLL with `LoadLibrary`. This approach leaves behind easily detectable artifacts such as newly loaded modules in the victim's virtual memory [6], [35]–[37]. While it allows for more complex code than shellcode, it may fail if the DLL is unsigned [6], [7].

Another approach is reflective DLL injection, which creates a custom loader in the DLL itself. The attacker maps the DLL sections into the remote process memory directly, and then the attacker invokes the custom reflective loader entry point, avoiding `LoadLibrary` usage. Reflective DLL injection avoids using module-lists and may not be caught up by EDR hooking or scanning at load time [6], [7], [35].

Process hollowing is a popular process injection technique. The attacker starts the process in a suspended state. Next, the original section is unmapped and replaced by attacker-controlled code. After resuming, the target process continues with the same process name but now executes the new code. This approach is often used by malware families to hide behind familiar Windows binaries and avoid inspection [6], [35]–[37].

Hooking injection can be done with WinAPIs such as `SetWindowsHookEx`, which registers a callback procedure that executes in another process' context whenever certain system events occur, like key presses or mouse movements. The attacker can attach a malicious DLL with a hook procedure in the victim process, so when the system event occurs, the malicious DLL is loaded into the remote process [6], [35].

Configuration injections use `IFEO` or `AppInit DLLs` with registry entries that cause Windows to load a malicious DLL. For instance, a DLL can be registered with `AppInit DLLs`, causing Windows to load it into processes that link to user32.dll. In this case, it is the OS that loads the DLL, helping to avoid EDR that is monitoring only for remote memory writes or thread creation [6], [35]. This approach is effective at stealth but requires administrative rights, and it can be blocked.

Some attackers also use `.NET` injection, which targets the Common Language Runtime (CLR). Instead of hooking or calling lower-level APIs directly, the attacker manipulates `.NET` assemblies in memory, loading or redirecting them in a CLR environment. This approach avoids EDR that checks for loaded PE files, as managed code layouts differ from those belonging to PE files [35].

PE Injection allows a PE file to be injected into another

active process. It then manually rebuilds or relocates its sections before redirecting execution. This is similar to process hollowing but does not require the unmapping of an existing image. Instead it simply places the new code in the target process and adjusts its entry point, and it creates a remote thread to execute the injected PE [6], [37].

Thread Execution Hijacking involves suspending an existing process and then unmapping its memory. Instead of creating a new thread, attackers pause an existing thread in the victim process and modify its register context, and typically they point the instruction pointer to injected code. As it does not create a new thread object, this approach can subvert some EDR monitors that track `CreateRemoteThread` calls, making it more stealthy [6], [37].

Asynchronous procedure call (APC) injection uses `QueueUserAPC` to place code in the APC Queue for a thread in a process. If said thread enters into an alertable state, it can then execute a code address that was set with the APC function. As threads in alertable waits regularly appear in real-world processes, APC injection can often be effective. As this approach uses legitimate APC capabilities instead of memory allocations it leaves less IOCs [6], [37].

Klein and Kotler [7] described injection sub-techniques that focus on mapping sections, like `NtMapViewOfSection`, rewriting kernel callback tables, or hooking window structures. Other techniques addressed in [7] and [34] also have noteworthy features:

- NtMapViewOfSection: The attacker maps a file-backed section, often from the pagefile, into both the injector and the target, writes the payload locally, then *forces* that section view into the target. This avoids using `VirtualAllocEx` + `WriteProcessMemory`.
- PROPagate: Overwrites a subclassed window object so that its function pointer leads to malicious code. Sending a window message triggers execution.
- NtMapViewOfSection-based Injection: Maps a file or the pagefile into both processes and writes the payload locally, so that the same data appears in the target memory without calling `WriteProcessMemory`.
- Ctrl-Inject: Overwrites the SingleHandler callback for `Ctrl-C` in a console app; once SendInput triggers a Ctrl-C, the callback in the target is malicious code.
- Thread Local Storage (TLS): Attackers exploit the per-thread storage mechanism by either adding or modifying callbacks and data in the TLS area, causing malicious code to execute under normal thread operations.
- Ptrace System Calls: On Unix, `ptrace` allows a process to attach to and debug another, so an adversary can attach to the target process, inspect its memory, and overwrite instructions or data to perform code injection.

MITRE [34] and Klein and Kotler [7] provide extensive detailed information on various process injection attacks. While the latter listed known variants, the MITRE ATT&CK framework also tries to classify each; this framework has become the gold standard for distinguishing between different tactics and techniques used by adversaries, focused exclusively on what is done in the wild. Starink et al. [6] grouped process injection techniques into 17 categories. It revealed that many traditional methods of process injection remained widespread, while there is also a trend towards novel techniques focused on evasion to remain undetected by EDR. Recent research, e.g., [38] indicates that there are currently gaps in EDR detection, which can include removing user-land hooks, and they note ROP activities are outside the scope of EDR, making detection less likely to be possible. This research is not intended to explore all process injection variants.

### B. ROP USAGE IN PROCESS INJECTION

While this paper presents the first generalized methodology for complete process injection via ROP, earlier techniques such as Stackbombing and Ghost-writing previously demonstrated limited but clever uses of ROP to support process injection. These prior methods primarily utilized brief snippets of ROP for a small number of specific tasks, relying on shellcode or higher-level code, such as C/C++ and C#, to implement most of the process injection. We are not aware of any other attempts, other than our own, where a vast majority or all of the tasks of process injection are implemented directly via ROP. We examine a few examples of ROP being used directly as part of process injection.

The banking malware Dridex [39] was the first malware sample observed using the AtomBombing code process injection technique, helped minimally by a ROP chain. In this method, the malware writes its payload into the Windows global atom table, using `GlobalAddAtom`. This code later could be retrieved by the `GlobalGetAtomName` thereby copying the malicious code into the target's memory. Next a ROP chain is used to bypass DEP by creating a region of memory that was RWX. The ROP chain also was used to copy the injected code from the RW section to the newly created allocation, where it was then executed. Overall, the ROP usage here was minimal.

Stackbombing [40] suspends a victim thread and saves its stack pointer (`RSP`). A small ROP chain is then placed at the top or in an unused portion of the stack by calling `NtQueueApcThread`. Afterwards, the attacker overwrites the function's return address so that, once the function returns, execution pivots to the ROP chain. This ROP chain performs a handful of tasks, such as copying shellcode into the target's memory and temporarily modifying page protections. Then it restores the stack pointer to its original value, resuming normal execution. In practice though, only a few WinAPI calls are made via ROP, for thread manipulation or memory copying. The bulk of process enumeration, privilege escalation, and advanced tasks are still implemented via higher-level code. Stackbombing has minimal ROP usage, using ROP for a specific purpose in a limited manner. Stackbombing is a complex attack that evades detection, but it has only sparingly been used as a proof of concept.

This limited ROP usage is also evident in Pinjectra, a C++ library introduced by Kotler and Klein [7], which implements

various process injection methods, including stackbombing. Yair [41] extended Pinjectra to include a few additional WinAPIs and Windows syscalls implemented via ROP.

Ghost-writing [7], [9] similarly use partial ROP; they do so by hijacking a suspended thread's context via `SetThreadContext`, leaving the thread to run in an infinite loop This loop continues until the full malicious payload is assembled. It uses ROP in conjunction with `NtQueueApcThread` and `memset`, injecting malicious code. Additional ROP gadgets are then used to restore registers, allowing execution to resume. This technique shows how only a partial ROP pivoting sequence can help perform code assembly in memory. While Ghost-writing illustrates how minimal ROP can stealthily inject code, the ROP continually pivots the thread context repeatedly to invoke a single memory-write gadget, to build the payload and the fake call frame. Similar to Stackbombing, nearly all other tasks are performed by higher-level code, including tasks such as enumerating processes, adjusting token privileges, or deciding which thread to hijack.

Although Stackbombing and Ghost-writing both illustrate creative ways of pivoting to ROP or using ROP for certain minimal tasks, they do not provide a comprehensive ROP-only injection. They decide not to use ROP to handle more complex injection steps, such as enumerating target processes, capturing a PID, allocating remote memory, and creating threads. In contrast, with our approach, everything occurs via a unified, multi-API chain; no higher-level code or shellcode is required to "fill in the gaps." Ghost-writing's use of ROP is primarily to perform repetitive micro-pivots to invoke a single write gadget many times, incrementally building the payload. As a result, our work not only demonstrates a fully ROP-driven injection but also generalizes it by providing a reusable library of ROP patterns, which can be used to implement all injection tasks entirely via ROP chains, eliminating the need for higher-level code or using shellcode as an intermediary to implement shellcode. In addition, our method fully addresses practical considerations of implementing a multi-API process injection using only ROP, thus bridging the gap left by previous work. This level of completeness and the portability of our work differs markedly from Stackbombing's and Ghost-writing's limited, though highly effective, usage of ROP. This also makes clear our work's emphasis on pushing the real-world boundaries of what has been done with ROP on Windows.

### C. PROCESS INJECTION DETECTION

Nasereddin and Al-Qassas [35] proposed a method for memory forensics to detect fileless malware and process injection attacks. Their approach uses parallel live memory analysis and Windows Event Tracing to identify artifacts, such as RWX pages, suspicious unbacked modules, thread start addresses, or altered registry keys, with the goal of catching attacks like DLL injection and process hollowing before they write files to disk. Their proposed tool was designed to monitor memory and catch anomalies in running processes; they were able to detect six process injection variants, including reflective DLL injection, registry-based injection, and process hollowing. Their focus was on detection and analyzing memory to discover malicious code, whereas our work explores ROP-only process injection, showing how ROP techniques can achieve process injection, while minimizing footprints often monitored by EDR.

## III. METHODOLOGY

### A. OVERVIEW OF THE APPROACH

On Windows, ROP is often used as a method to bypass DEP, thereby allowing shellcode to be executed; exploits relying on shellcode are well-established techniques and are less time-intensive to implement. A close examination of available ROP code on ExploitDB [19] and other public sources confirms this observation. While ROP on Windows has far more potential–and ROP has been shown to be Turing-complete on most architectures–that full potential may not necessarily be realized on every binary. Certain Turing-complete features, such as conditionals, can be more challenging to set up and use in ROP, and while they may be possible on some binaries, it would be erroneous to assume such functionality is present in all binaries. More to the point for this research, with process injection as our case study, a lack of preexisting patterns for the majority of WinAPIs would make constructing process injection—or any equally complex chain of many WinAPIs—monumentally difficult and time-consuming. In the vast majority of cases where ROP is used on Windows, the focus is on using a prebuilt pattern in conjunction with PUSHAD to invoke one of a small set of WinAPIs. Real-world instances of conditional logic are rare. Thus, real-world ROP usage has not reached its full potential.

This work uses ROP to accomplish a complete process injection workflow that does not rely on shellcode or any higher-level language. This is the first and only work that allows for process injection to be done entirely via ROP. We have developed a chain for up to thirty Windows APIs (WinAPIs), some of which may be repeated, covering tasks such as dynamic memory allocation, privilege escalation, and payload execution. In contrast, public examples almost always rely on one API call, or at most two or three. This research advances the field of offensive security by achieving the entire process injection workflow exclusively via ROP, and because it uses only ROP, it is more resilient against EDR.

Process injection was selected as a case study because it spans numerous steps, from locating an appropriate target process to creating and executing a remote thread inside that process. Thus, it is an advanced chain of APIs, requiring that many handles be acquired and the construction of various Windows structures in memory, each of which may be used as input or receive output. Real-world implementations of process injection typically rely on code written in higher-level languages for most of these stages and only rarely use ROP for limited tasks. Our approach departs from the status quo by invoking each required WinAPI and associated

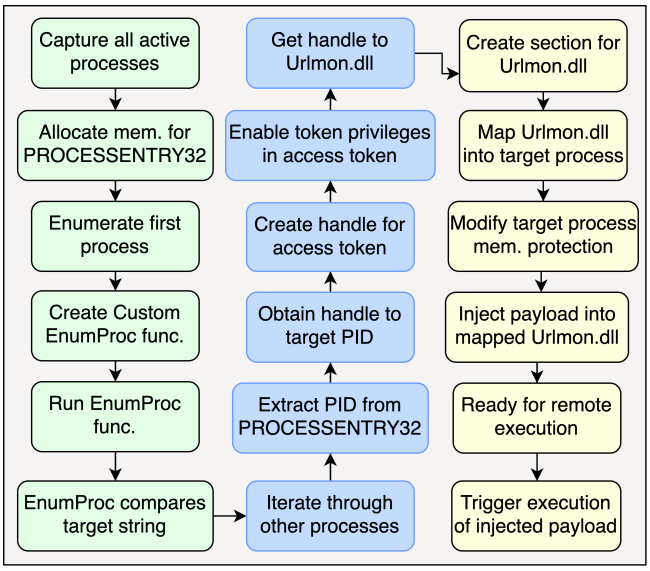


**FIGURE 1.** The overall methodology for ROP-based process injection.

parameters via ROP alone. For this to work, we rely on a entirely original library of parameter-loading patterns. These patterns define which values for required WinAPI parameters must be loaded into which registers, so that once the PUSHAD instruction is used, all the registers are pushed onto the stack in a predetermined order, allowing that target API to be called. Our process injection case study, thus, is also potentially replicable across multiple binaries, due to the many novel patterns.

The overall methodology is depicted in Fig. 1, and pseudocode is shown in Algorithm 1. One of our ROP process injection techniques can be represented via pseudocode, although there are many opportunities for variations, to minimize use of functions such as `VirtualAlloc` and `VirtualProtectEx`, for extra stealth. Here `ROPInvoke` stands for the sequence of ROP gadgets that are used to load the parameters, using registers or the stack, and then executing either a PUSHAD or stack pivot, before then jumping to the target API's runtime address, such as with a `JMP [REG]` gadget.

Our method demonstrates that by using our library of novel patterns and our custom `EnumerateProcesses` function, ROP can manage all phases of an advanced process injection workflow, realizing the full potential of ROP. As mentioned, this approach incorporates reusable ROP patterns, or blueprints, for individual API calls, each often with many variations. Because this work is not done on a single binary but is designed to be generalizable through its set of patterns, it can be replicated on potentially a large number of binaries. This research also overcomes the challenge of performing repeated string comparisons via ROP—a prerequisite for identifying and targeting processes—through the creation of a custom `EnumerateProcesses` function. Implemented entirely via ROP, this function iterates through active processes, matches the target process by name, and retrieves its PID, integrating it into the process injection workflow.

### B. CHALLENGES ADDRESSED

Implementing process injection solely through ROP involved overcoming a variety of challenges. These included the practical limitations of ROP on modern Windows platforms, as well as the need to create an approach that could bypass modern defenses. One challenge was the inherent complexity of chaining together multiple WinAPIs [5] since some WinAPIs required one or more handles that could only be generated as return values from other WinAPIs. Additional challenges included performing string comparisons via ROP and addressing the possible scarcity of certain gadgets, given that each binary's attack surface may differ.

One challenge presented by this research was developing an approach for chaining multiple WinAPIs entirely via ROP. Each WinAPI call requires a unique setup of parameters in specific registers, which must not be clobbered until PUSHAD is executed. The PUSHAD instruction imposes strict requirements on how parameter values are loaded. To overcome these constraints, multiple reusable ROP patterns were developed for each WinAPI, making it possible to chain together more than 30 WinAPIs in the case study. Beyond just chaining gadgets, coordinating dozens of WinAPIs demands careful handling of handles, return values, and out parameters. Because many WinAPIs on Windows require one or more handles from other WinAPIs, these values must be saved to memory and retrieved as needed—potentially much later in the ROP chain. Although this is straightforward in higher-level languages like C or C++, it can be far more challenging when relying solely on ROP gadgets.

String comparisons posed another hurdle, as identifying the correct process for injection requires iterating through active processes and matching each name to a target string. Although trivial in a higher-level language, within ROP it demands a gadget or chain of gadgets that can perform character-by-character comparisons. The scarcity of such gadgets in many binaries, combined with the need for repeated comparisons, makes this especially difficult. In many binaries, it would not even be feasible due to insufficient gadgets. However, once the correct string is found, the PID can be retrieved and converted into a handle by a subsequent WinAPI, enabling injection. Without that PID, an attacker might attempt to inject into random or privileged processes, which is not viable. Capturing the PID is thus a requirement, yet fulfilling it can involve hundreds of comparisons. Moreover, additional handling is needed once the target string is found. This research overcomes these obstacles by creating a custom `EnumerateProcesses` function, allowing for ROP-based string matching and PID extraction.

Finally, this research was intended to demonstrate the full capabilities of ROP on the Windows platform and provide an open-source reference for future work. Simply crafting a complex ROP chain for one binary would have limited practical value, because the approach and gadgets would be specific to that binary. Such non-replicable work would offer

**Algorithm 1** ROP_Process Injection Using NtMapViewOfSection

```
1: Algorithm ROPProcessInjection(targetProcName)
2: hSnapshot ← ROPInvoke(CreateToolhelp32Snapshot, TH32CS_SNAPPROCESS, 0) ▷ Capture snapshot of all running processes
3: memAddrPE32 ← ROPInvoke(VirtualAlloc, NULL, SIZE_OF_PROCESSENTRY32, MEM_COMMIT, PAGE_EXECUTE_READWRITE) ▷ Allocate memory for PROCESSENTRY32
4: ROPInvoke(Process32First, hSnapshot, memAddrPE32) ▷ Initialize PROCESSENTRY32 with first process
5: memAddrROPCustomFunc ← ROPInvoke(VirtualAlloc, NULL, SIZE_OF_ENUM_FUNC, MEM_COMMIT, PAGE_EXECUTE_READWRITE) ▷ Build & write custom function to enumerate process to memory
6: WriteCustomEnumProcFunc(memAddrROPCustomFunc) ▷ This calls Process32Next repeatedly, comparing names. ▷ On success, the PID is stored at memAddrPE32 + 0x24.
7: ROPInvoke(EnumProcFuncLoop, memAddrROPCustomFunc) ▷ Run the enumerator loop, searching search for targetProcName
8: targetPID ← ReadMemory(memAddrPE32 + 0x24) ▷ Extract the target PID from PROCESSENTRY32 struct
9: hTargetProc ← ROPInvoke(OpenProcess, PROCESS_ALL_ACCESS, FALSE, targetPID) ▷ Convert PID of target process to a handle
10: hToken ← ROPInvoke(OpenProcessToken, CURRENT_PROCESS, TOKEN_ADJUST_PRIVILEGES) ▷ Obtain token to target process & set SeDebugPrivilege
11: ROPInvoke(AdjustTokenPrivileges, hToken, EnableDebugPrivilegeStruct)
12: hFile ← ROPInvoke(CreateFileA, "C:\\Windows\\SysWOW64\\urlmon.dll", GENERIC_READ, ...) ▷ Create a handle to file for urlmon.dll
13: ROPInvoke(NtCreateSection, &hSection, GENERIC_ALL, NULL, NULL, PAGE_READONLY, SEC_IMAGE, hFile) ▷ Create a section backed by urlmon.dll
14: ROPInvoke(NtMapViewOfSection, hSection, hTargetProc, &lpBaseAddr, NULL, NULL, NULL, &viewSize, ViewShare, NULL, PAGE_READONLY) ▷ Map a section backed by urlmon.dll
15: ROPInvoke(VirtualProtectEx, hTargetProc, lpBaseAddr, regionSize, PAGE_EXECUTE_READWRITE, &oldProtect)
16: ROPInvoke(WriteProcessMemory, hTargetProc, lpBaseAddr + offset, localPayloadBuf, payloadSize, &bytesWritten) ▷ Write the malicious payload
17: ROPInvoke(CreateRemoteThread, hTargetProc, NULL, 0, (lpBaseAddr + offset), NULL, 0, &dwThreadId) ▷ Start thread in target process ▷ Payload executes inside targetProcName
18: return Success
```

minimal benefit to the security community. To generalize this approach and make it repeatable across a range of binaries, a library of interchangeable patterns was created for each WinAPI, maximizing the ways ROP can be applied to different binaries. If one pattern proves difficult to use, another may be substituted. Since more than one pattern is provided for each WinAPI, where possible, the methodology is not dependent on a single pattern or rigid sequence of gadgets.

### C. DEVELOPMENT OF ROP PATTERNS

To create a process injection workflow entirely dependent on ROP, at least one pattern is needed for each WinAPI when using the PUSHAD instruction, which is the most common approach in Windows. Patterns define which values must be placed into specific registers so that, when PUSHAD is executed, the WinAPI is called with correctly ordered parameters and a valid return address. These patterns are established through trial and error in a debugger, such as WinDbg, by making careful observations regarding outcomes.

ROP in Windows is almost invariably tied to using WinAPIs to be an agent of change. Historically, this has mainly been done to bypass DEP, although any WinAPI could be used for any purpose. While common WinAPIs such as `VirtualProtect` and `VirtualAlloc` have had all their patterns documented—two each—the fact that so few others are seldom used, means there is not any kind of library of such patterns. On those uncommon occasions when a WinAPI's pattern is undocumented, an attacker or researcher typically creates it from scratch. If a WinAPI's pattern is documented, then it can simply be followed.

How to create patterns has never, to our knowledge, been documented, and as it is technical and low-level, doing so often would be avoided unless necessary. Each pattern differs according to the number of parameters a WinAPI requires. For instance, a WinAPI with three parameters might support multiple patterns, and another WinAPI with the same number of parameters may be able to reuse some patterns with minor modifications.

Our patterns are based primarily on the PUSHAD instruction, which pushes registers onto the stack in a predefined order: `ESI`, `EDI`, `EBP`, `ESP`, `EBX`, `EDX`, `ECX`, `EAX`. This order is useful for calling functions with their parameters in the correct order. After PUSHAD, whatever was in `ESI` executes as a ROP gadget. One common feature of all these patterns is that once a WinAPI is invoked, the stack is set up with both the return address and the function parameters at the top. This can be achieved in multiple ways. One way to invoke a WinAPI is to place a pointer to a WinAPI in a register, and then a dereferenced jump. As it can be assumed that ASLR will protect all WinAPIs, it is not possible to know its runtime address prior to execution. However, many pointers to WinAPIs exist on binaries; pointers can be used to allow a WinAPI to be used regardless of ASLR. With `JMP DWORD PTR [EBX]`, `EBX` could be used to dereference the runtime address and cause it to be executed. In that case, `EDX` would hold the return value, and `ECX` and `EAX` would be parameters, as those registers follow `EBX`. Although this is ultimately just control-flow manipulation that follows standard calling conventions, the way we populate the stack with the return address and parameters is unusual—no compiler would generate using PUSHAD as a way to cause an API to be invoked. After the invocation, any misalignment would cause the WinAPI to fail, so patterns that deviate would not survive testing, and thus the pattern candidate would be rejected.

In developing patterns, we use seven of the eight registers, generally excluding `ESP`. `ESP` is problematic for parameters unless a parameter specifically needs to point into the stack.

In most patterns, `ESP` is skipped. One common method to skip the `ESP` register is by placing a `RET 4` instruction in the second register before `ESP`. This approach allows the pointer in the next register to execute while skipping the following one, which could be `ESP` in this case. This technique successfully skips the `ESP` register, although our patterns present many other ways of skipping `ESP`. In most cases, it is important to both skip loading `ESP`, as well as to avoid having the pointer in `ESP` being executed.

Having more than one pattern for each allows for the approach to be portable and not tied simply to one specific binary. Through experimentation and testing, we created as many patterns as possible for each WinAPI. While the assignment of parameters and return values is important in patterns, the means of how they are loaded have no requirements. The values for the patterns can, in fact, be loaded in a virtually unlimited number of ways; there is no order to when values must be loaded. As pattern values are loaded into their registers, values previously loaded into registers must be preserved and not clobbered.

Each pattern was validated in multiple test environments across various Windows platforms. Therefore, we can use these patterns to reliably invoke the WinAPI that each pattern targets. Another way to call a WinAPI is by writing parameters and the return address directly to the stack, then performing a stack pivot so that the WinAPI is invoked with the correct arguments. This method uses MOV dereferences, such as `MOV DWORD PTR [EDI], EBX`, where `EBX` might hold a function parameter and `EDI` points into the stack area. Gadgets like `INC EDI` or `DEC EDI` can advance `EDI` until all parameters, the return address, and the WinAPI's dereferenced runtime address are positioned correctly. A stack pivot gadget could be used to move `ESP` so that when the next `RET` is executed, execution pivots to the WinAPI. This technique has been called the sniper technique [42]; in this paper, we will refer to it both as the sniper technique and as the MOV dereference technique. PUSHAD does not exist in x86_64 Assembly, and the 64-bit calling convention requires arguments to be loaded first into `RCX`, `RDX`, `R8`, `R9`, and then the stack. This research does not address creating patterns for x86_64, since PUSHAD is not available, and loading registers is more straightforward in x86_64, whereas it is very non-intuitive with x86.

Although the sniper technique avoids PUSHAD, it requires significantly more ROP gadgets, and so it is often avoided. When using MOV dereference by itself, generally we could only develop one pattern for it. However, in this research, we also pioneered the use of combining the sniper approach with the PUSHAD technique. This can be effective for WinAPIs where there are more parameters than can be accommodated with only the PUSHAD technique. Thus, values are loaded into patterns for some of the required parameters, while the sniper approach can be used for the remaining parameters. In these cases where parameters exceed the number of registers available using the PUSHAD technique, we combine these approaches, as it still requires significantly fewer ROP gadgets, and payload size is not unlimited.

### D. API CHAINING METHODOLOGY

Having multiple patterns for each WinAPI is not enough; the process injection workflow requires a way to unite these patterns into a cohesive chain, so that all steps of process injection can be achieved, including process enumeration, adjusting privileges, injecting the payload, and remotely executing the payload. This subsection details how the ROP patterns can be used to create a complex chain of WinAPI calls.

As noted previously, most ROP chains on Windows simply bypass DEP, typically using only one WinAPI, while an extremely limited number might use up to three [3]. Having more than three on Windows almost never is done on Windows. Because this work can involve chaining up to 30 WinAPIs, it demands more specialized handling. In our case study, we also note that some WinAPIs, such as `GetProcAddress` and `LoadLibraryA`, were called more than once.

Each pattern has been carefully validated before being inserted into the larger chain. For example, `VirtualAlloc` must allocate a region of memory with the desired permissions, and the pattern for `CreateToolhelp32Snapshot` must return a valid handle to the snapshot used for enumerating processes. Once these patterns work in isolation, they can be chained together to complete the various tasks of process injection.

Using a single WinAPI, such as to bypass DEP, can be relatively straightforward and may not require much special handling. However, chaining multiple WinAPIs raises special considerations. For example, many of these WinAPIs are used specifically to obtain resources like handles, which are often provided in return values or sometimes in-out parameters. If a value is stored in `EAX` and not immediately transferred elsewhere, it could be lost once a different WinAPI with a different pattern is invoked. To avoid this, our approach builds a cache in memory—often on the stack—where values can be saved and retrieved as needed. By carefully preserving the output from previous WinAPIs and aligning it with the input requirements of the next, the chain remains stable.

Managing many different handles and return values, which may be required by multiple WinAPIs, calls for careful planning to ensure these resources can serve as parameters for subsequent calls. Because multiple values might need preserving before being overwritten, a frame of reference must be maintained throughout the ROP chain. For example, one could create a cache by capturing a pointer to `ESP` in `EBX` and adding or subtracting as needed, to create a memory point of reference. Regardless of which register holds this reference, the register must eventually be preserved—perhaps by pushing it onto the stack at a designated spot and retrieving it later. Where and how values are stored often varies throughout the exploit, depending on each WinAPI's nuances. Another option is to skip using a fixed reference and instead calculate

the exact distance to preserved parameters, though this is less efficient, as offsets can change frequently.

When each WinAPI concludes, execution passes to its return address. Since this approach is based on ROP, that return address must be handled carefully. Two options exist: The first is to ignore the return address by having it return to an ROP nop—essentially a gadget that leads to a `RET`—allowing execution to continue. The second option is to make the first gadget that prepares the next WinAPI the return address, thus flowing directly into the next chain segment. If return values or out parameters must be preserved, the first gadget responsible for saving them can serve as the return address. In any case, the return address must be planned with care.

By carefully managing return values and out parameters and making sure that execution flows from one WinAPI to the next, the chain can use a potentially unlimited number of APIs, limited only by any payload-size restrictions. Chaining WinAPIs together is needed so that the full spectrum of the process injection workflow can be united into one cohesive sequence, from process identification, privilege escalation, payload injection, and remote execution.

Overall, the entire ROP sequence offers significant flexibility in how multiple WinAPIs can be chained. Because we have generated multiple patterns for most WinAPIs, how to construct the chain is flexible. Any pattern can be used interchangeably with others for the same WinAPI, provided there are sufficient gadgets. In general, wherever possible, the PUSHAD technique can be used with individual WinAPIs, reducing the number of gadgets required. However, the complete chain can also accommodate WinAPIs whose patterns require the hybrid approach of using both PUSHAD and the `mov` dereference approach. As a final option, `mov` dereferences can also be used in isolation for sections of the chain, if PUSHAD is not feasible, though the mechanics of its approach require significantly more gadgets.

### E. CUSTOM ENUMERATEPROCESSES FUNCTION

In standard ROP exploits, a common limitation is the difficulty of handling non-trivial logic such as string comparisons, loops, and conditionals. These usually require many specialized gadgets; often these gadgets are not present. And, one of the main challenges preventing process injection in ROP is the lack of reliable string-comparison functions that can be integrated into a looping conditional. The combination of such gadgets to reliably do so, and to be available as gadgets on perhaps any given binary, was thought to be unlikely. As this work is intended to be portable and function purely on ROP, creating a custom `EnumerateProcesses` function was the best solution. The `EnumerateProcesses` function is a needed component of the overall injection workflow, as without it likely there would be insufficient gadgets on many binaries to complete these steps using only ROP. While imperfect, it provides a universal solution to the string-comparison problem, i.e., that many binaries do not provide suitable gadgets. Certainly, if a given binary has the right gadgets, they could be used instead, but this function provides an alternative. In addition, the custom `EnumerateProcesses` function illustrates a broader concept of ROP exploitation. With the `EnumerateProcesses` function, the exploit chain is no longer limited by the availability of `cmp, repz cmpsb`, or branching gadgets. Whenever a more advanced task is unavailable or there are insufficient gadgets, new functions can be created and used as an ROP gadget.

### F. HANDLING FAILURE

Because our approach relies entirely on ROP, handling errors (e.g., unsuccessful API calls or insufficient privileges) is not straightforward. In contrast to higher-level languages, where it is easy to handle exceptions or implement status checks, with ROP this would be challenging to implement. The attack surface for ROP often lacks practical, convenient gadgets for conditionals or branching, so implementing error-handling or retry logic via ROP is impractical. Consequently, if a particular call fails, the exploit is not able to detect it and self-correct, and the overall process injection attempt will fail.

In a higher-level language, an API failure might be caught via exception handling or status checks; in a ROP chain, such checks would require specialized gadgets that do not always exist in every binary. Still, throughout our year-long experimentation, the parameter values we have devised and selected have proven reliable. In the binaries tested, the selected parameters and the particular chain sequence succeeded consistently. It is important to note that if the chain is set up and debugged for a specific application and environment, it continues to work reliably in similar environments. In addition, insufficient privileges can still cause the technique to fail in some cases. For instance, if the target process belongs to a higher-privileged user, calls like `CreateFileA` could fail. One of our chain variations includes `AdjustTokenPrivileges`, specifically to resolve this problem via privilege escalation. When needed, this approach can be extended to other chain variations if required.

Our generalized approach and sample chains serve as a strong, highly reliable starting point, but developers who adapt our approach for different applications must be prepared to fine-tune certain parameter values via debugger. Developing ROP in practice involves trial and error with debuggers; however, once the ROP chain, comprised of multiple WinAPIs, works for a given binary and environment, it will usually continue to be stable. Additionally, for each individual WinAPI or native API supported, we have provided all available parameter-loading patterns, offering flexibility. The patterns for individual WinAPIs are highly likely to yield successful outcomes with suitable values. While the individual WinAPIs may be effective, it is also necessary to ensure that handles generated by one WinAPI are handled safely and are passed to subsequent WinAPIs.

One potential limitation is gadget availability. If the necessary gadgets for pushing parameters or invoking APIs do

**TABLE 2.** Different patterns are depicted for the CreateToolhelp32Snapshot (CT32S) WinAPI.

| Reg. | Pattern 1 | Pattern 2 | Pattern 3 | Pattern 4 | Pattern 5 | Pattern 6 | Pattern 7 | Pattern 8 | Pattern 9 | Pattern 10 | Pattern 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| EDI | RET 8 | Pop | Pop | ADD ESP, 0xC | ADD ESP, 0xC | ADD ESP, 4 | ADD ESP, 0xC | ADD ESP, 0xC | RET 4 | ADD ESP, 0xC | ADD ESP, 4 |
| ESI | RET | CT32S PTR | CT32S PTR | RET | CT32S PTR | CT32S PTR | ADD ESP, 4 | RET 4 | ADD ESP, 4 | Pop | CT32S PTR |
| EBP | RET | Pop | ADD ESP, 4 | CT32S PTR | RET | ADD ESP, 4 | CT32S PTR | CT32S PTR | CT32S PTR | CT32S PTR | Pop |
| ESP | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip |
| EBX | CT32S | Jmp [ESI] | Jmp [ESI] | Jmp [EBP] | Jmp [ESI] | Jmp [ESI] | Jmp [EBP] | Jmp [EBP] | Jmp [EBP] | Jmp [EBP] | Jmp [ESI] |
| EDX | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address |
| ECX | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags | dwFlags |
| EAX | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID | th32ProcessID |

not exist in a specific binary, then one of the alternative patterns should be used. Our library of nearly 150 parameter-loading patterns provides flexibility by offering multiple ways to load registers, which can help overcome gadget scarcity. Ultimately, a vast majority of gadgets required for these patterns are gadgets that load values into a register, and there can be virtually endless ways to achieve that. For instance, instead of using `pop edx`, we might instead use an integer overflow, adding values into registers that–when added together–will produce the desired value in `EDX`. Very often creativity can help find alternative ways to load values into specific registers; what ideally could be done in one gadget may require several gadgets. These specific gadget combinations are outside the scope of this work. If there are sufficient gadgets to load registers for one pattern, then it is highly likely that there are sufficient gadgets for all registers. In rare cases, if the attack surface is highly limited, then it may be necessary to attempt one of the other process injection methods we support. In cases with extreme gadget scarcity, process injection via ROP may not be possible.

## IV. IMPLEMENTATION

Building on the methodology described earlier, this section details the API-specific implementations and discusses novel contributions in pattern design. Using our ROP-based process injection approach, we demonstrate how 15 WinAPIs can be used for advanced functionality, including process enumeration, memory allocation, privilege escalation, payload injection, and execution. In practice, some APIs are repeated, including helper WinAPIs, resulting in over 30 invocations—about a tenfold increase from the largest number of WinAPI calls previously observed. These WinAPIs form the backbone of the process injection workflow, each implemented by meticulously constructed ROP chains following our methodology.

### A. CREATETOOLHELP32SNAPSHOT

The `CreateToolhelp32Snapshot` API captures a snapshot of all processes running on the system, forming the basis for identifying the target process. Furthermore, this WinAPI provides a handle to the snapshot, which is needed by later enumeration WinAPIs like `Process32First` and `Process32Next`. Both rely on this snapshot handle to iterate through processes until the target is located. The goal of using `CreateToolhelp32Snapshot` is to obtain a handle called `hSnapshot`, which contains detailed information about all active processes. Implementing this WinAPI via ROP involves multiple stages. First, the `dwFlags` parameter can be set to `TH32CS_SNAPPROCESS` to include process information, and `th32ProcessID` can be set to 0, specifying the current process context. By setting these through ROP, we gain a handle on the snapshot, which is a key part of the process injection workflow; without it, no further progress would be possible. If a pointer to `CreateToolhelp32Snapshot` is available, it can be used directly. If not, `GetProcAddress` is invoked dynamically to resolve its address. Once parameters are loaded and the API's runtime address is known, the chain executes the WinAPI. The handle to the snapshot returns in `EAX` and can be stored for later ROP chains in what we call a cache, a predefined stack location to preserve values.

We developed eleven patterns for invoking CreateToolhelp32Snapshot, each thoroughly tested and summarized in Table 2. All patterns rely on `PUSHAD` to load the required values into the registers they are mapped to. The ROP chain begins by using loading gadgets to place these values appropriately. Once all registers are set, `PUSHAD` pushes them onto the stack, causing `CreateToolhelp32Snapshot` to run. In some patterns, a gadget like `JMP DWORD PTR [EBX]` is used to jump to the WinAPI, with its return address and parameters aligned on the stack. Once it finishes, `hSnapshot` resides in `EAX`. The success of this WinAPI is confirmed when `hSnapshot` returns in `EAX`. We extensively tested the `CreateToolhelp32Snapshot` ROP chain across multiple Windows versions, verifying that valid process snapshots were created for each of the eleven patterns. This interchangeable design demonstrates the portability of our approach, allowing it to be used on multiple binaries with different attack surfaces.

### B. VIRTUALALLOC

`VirtualAlloc` allocates memory in the target process with the desired permissions, creating a region to store the payload. For process injection, this function is used to allocate memory for the `PROCESSENTRY32` structure that `Process32First` will create. While the stack could store this structure, using separate memory is simpler if there are no strict restrictions on payload size. The purpose of `VirtualAlloc` is to reserve and commit memory regions in the target process. Although other options are possible, the memory can be allocated with the `PAGE_EXECUTE_READWRITE` protection attribute, allowing it to be written to, read, and executed. This same memory may also host a custom function later, hence it is made

**TABLE 3.** Two different patterns are shown for the VirtualAlloc WinAPI.

| Register | Pattern 1 | Pattern 2 |
|---|---|---|
| EDI | RET | RET |
| ESI | VirtualAlloc | Jmp [EAX] |
| EBP | Return Address | Pop |
| ESP | IpAddress | IpAddress |
| EBX | dwSize | dwSize |
| EDX | flAllocationType | flAllocationType |
| ECX | flProtect | flProtect |
| EAX | Nop | VirtualAlloc PTR |

executable.

Implementing `VirtualAlloc` requires its parameters to be set up according to existing ROP patterns. For instance, `lpAddress` is set to `NULL`, letting the operating system choose the base address; `dwSize` is set to `0x128`; `flAllocationType` is set to `MEM_COMMIT`; and `flProtect` is set to `PAGE_EXECUTE_READWRITE`. In one pattern, `JMP DWORD PTR [EBX]` dereferences and jumps to `VirtualAlloc`'s runtime address, executing it with the correct arguments and return address. The WinAPI then returns the address of the newly allocated memory in `EAX`, which can be used in later process injection steps.

Unlike most WinAPIs in this work, `VirtualAlloc` already had documented patterns (see Table 3). After extensive experimentation, we concluded that only the two known patterns function reliably with the desired outcome. A `VirtualAlloc` pattern is considered valid if it returns an allocated memory address. Our validation confirmed that these two existing patterns work interchangeably on various Windows platforms.

### C. PROCESS32FIRST AND PROCESS32NEXT

`Process32First` and `Process32Next` iterate through the process snapshot, which was previously created by `CreateToolhelp32Snapshot`, handling one process at a time. `Process32First` populates the `PROCESSENTRY32` structure with detailed information like the process name, PID, and other process attributes; `Process32Next` advances to the next entry. Both APIs can allow the target PID to be captured by searching for its name. Traversing the snapshot to find the target process, thus, completes the enumeration phase of process injection.

For process injection, our purpose in using these APIs is to extract the `szExeFile` string from `PROCESSENTRY32`. Once this has been located, its corresponding PID can be captured, allowing for a handle to the process to be created by `OpenProcess`, so that code can be injected into our target process. Because there may be hundreds of processes active, a custom enumeration function, discussed in a later subsection, will be created to help discover the PID.

`Process32First` requires two arguments: `hSnapshot`, saved from `CreateToolhelp32Snapshot`, and `lppe`, a pointer to the `PROCESSENTRY32` structure, which has ten fields. The `dwSize` field can be set to `0x128`, or any arbitrary size, and the remaining fields are set to `NULL`.

As shown in Table 4, `Process32First` and `Process32Next` each have ten nearly identical patterns. The only difference is the pointer to the runtime address of each. Because there are just two parameters, all patterns require a method to skip certain values. As an example, `EDI` might be set to `RET 8`, and `ESI` and `EBP` both point to a ROP nop. Loading gadgets then place the two parameter values into designated registers: the handle to the snapshot goes into `ECX`, and the pointer to `PROCESSENTRY32` goes into `EAX`. Using ROP, the `PROCESSENTRY32` structure must be initialized by using a series of `MOV` dereference gadgets; the `dwSize` field is set to `0x128`, and all other fields are set to `NULL`. Once `PUSHAD` executes, a series of gadgets will follow, leading to the WinAPI being invoked; the return value is a Boolean indicating if the `PROCESSENTRY32` was updated.

If `Process32First` succeeds, then `PROCESSENTRY32` is updated with information about the first process in the snapshot. If `Process32Next` succeeds, it updates the structure with the next process's information. We validated each of the 20 patterns in `WinDbg` to confirm the return value was `TRUE` and that `PROCESSENTRY32` advanced correctly to the next entry. Any pattern for the corresponding WinAPI may be used, provided the attack surface supports it.

### D. CUSTOM ENUMERATION FUNCTION

We create a custom `EnumerateProcesses`function, without using shellcode, by writing its machine code bytes directly into a region of memory that has been made executable, such as `0x78000`. While we directly write these bytes into executable memory, we could also do it in a safer manner, by writing the code first, and then changing the code to make it executable. We begin with gadgets like `MOV DWORD PTR [EBP], ESI # RET` to copy each machine instruction byte to memory via ROP. We use four `INC EBP # RET` gadgets to advance `EBP`; this allows the remaining code to be written, placing a `double word` (DWORD) of bytes, one after the other, until the function has been totally writen. Finally, after `ESI` is loaded with the custom function's address, a simple `PUSH ESI # RET` instruction transfers control to our function and starts its execution of the function. Once fully complete, the function includes the following instructions:

```
enumerateProcesses:
mov ebp, edx
push dword ptr [ebp-0x800]
mov edx, dword ptr [ebp-0x7FC]
push dword ptr [edx]
mov edx,dword ptr [ebp-0x7F8]
call dword ptr [edx]
mov edi,dword ptr [ebp-800h]
add edi, 24h
mov esi,dword ptr [ebp-7F4h]
xor ecx,ecx
mov cl,8
```

**TABLE 4.** Different patterns can be seen for the Process32First/Process32Next (P32) WinAPI.

| Reg. | Pattern 1 | Pattern 2 | Pattern 3 | Pattern 4 | Pattern 5 | Pattern 6 | Pattern 7 | Pattern 8 | Pattern 9 | Pattern 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| EDI | RET 8 | Pop | Pop | ADD ESP, 0xC | ADD ESP, 4 | ADD ESP, 0xC | ADD ESP, 0xC | RET 4 | ADD ESP, 0xC | ADD ESP, 4 |
| ESI | RET | RET | RET | RET | RET | ADD ESP, 4 | RET 4 | ADD ESP, 4 | Pop | RET |
| EBP | RET | Pop | ADD ESP, 4 | RET | ADD ESP, 4 | RET | RET | RET | RET | Pop |
| ESP | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip |
| EBX | P32 | P32 | P32 | P32 | P32 | P32 | P32 | P32 | P32 | P32 |
| EDX | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address |
| ECX | hSnapshot | hSnapshot | hSnapshot | hSnapshot | hSnapshot | hSnapshot | hSnapshot | hSnapshot | hSnapshot | hSnapshot |
| EAX | Ippe | Ippe | Ippe | Ippe | Ippe | Ippe | Ippe | Ippe | Ippe | Ippe |

```
cld
repe cmpsb
jecxz match
jmp enumerateProcesses
match:
ret
```

ROP can also be used to initialize some of these memory locations, such as `ebp-0x800`, which is a pointer to a `PROCESSENTRY32` structure. These locations were stack addresses, which could be used reliably without issue. An alternative approach could be to designate registers to hold these values. The custom function prepares the necessary parameter values, loading the required parameters, the pointer to the `PROCESSENTRY32` structure and the `hSpanshot`, found at `ebp-0x800` and `ebp-0x7FC` respectively. The custom `EnumerateProcesses` function repeatedly calls `Process32Next`, having retrieved its runtime address from `ebp-0x7F8`, each time the function restarts. The function uses `REPE CMPSB` to compare each discovered process name to a target string, retrieved from `ebp-0x7f4`. If there is a match, it breaks out of the loop; otherwise, it returns to the function's start, repeating until the name is found. Once a match is found, the function exits. At that point, eight `INC ECX` gadgets reach the PID in `PROCESSENTRY32`; the PID is now available to be converted into a handle for the target process.

Because string comparisons are typically difficult under pure ROP, due to insufficient gadgets, embedding them in a custom function, comprised of a small piece of directly written machine code, allows us to implement looping and branching without needing specialized conditional gadgets. Getting such specialized gadgets to reliably implement the looping and branching via pure ROP would likely not be possible on a majority of binaries. However, by implementing this custom function entirely through ROP gadgets, we avoid shellcode and bypass the need for typical string-comparison instructions within the primary ROP chain itself, which would be unlikely to succeed. Our approach significantly improves the likelihood of successful process enumeration. Validation testing confirmed the reliability of this approach, as seen in Sec. V, with accurate matches observed in multiple test environments, as long as the target application was an active process. If the target process was not active, it would not be found.

Because `EnumerateProcesses` was designed specifically for tasks tied to process injection, it was not meant to be universal; it acts as a proof of concept for performing string comparisons via ROP. While this approach of creating a custom function could be applied to any problem that is impractical in standard ROP, doing so takes an extra burden. However, this does provide a practical, real-world solution to the problem of string-comparison being largely infeasible via standard ROP on Windows that utilizes gadgets found during exploitation.

### E. OPENPROCESS

The purpose of `OpenProcess` WinAPI is to grant access to the target process based on specified permissions, by providing a handle to the process. These permissions let the injecting process allocate memory, write data to the target's address space, and trigger the payload. The `OpenProcess` WinAPI obtains a handle to the target process for later operations, such as mapping a DLL into the target process and payload injection. Without this handle, work cannot continue beyond process identification. The needed parameters for `OpenProcess` are `dwDesiredAccess`, which defines the access level; `bInheritHandle`, which determines whether the handle is inheritable; and `dwProcessId`, which provides the target's PID. The PID is taken from the `PROCESSENTRY32` structure.

The ROP implementation involves loading registers with parameters according to any of the five patterns we created. As shown in Table 5, these patterns use a hybrid approach involving PUSHAD and `MOV` dereference to supply three parameters and the return address. Loading gadgets can be used to set the `dwDesiredAccess` parameter to `0x1F0FFF`, for `PROCESS_ALL_ACCESS`, providing full access to the target process. The `bInheritHandle` parameter is set to `0x0`, `FALSE`, as handle inheritance is not needed. The `dwProcessId` parameter is set to the PID taken from the `PROCESSENTRY32`, generated in the `EnumerateProcesses` custom function by using `Process32Next`. We will explore one of the patterns: `EDI` is set to `RET 8`, and `ESI` and `EBP` are both set to a ROP nop. `ESP` is skipped, and `EBX` is set to the runtime address of `OpenProcess`. The `dwDesiredAccess` value for `0x1F0FFF` is loaded into `ECX`, and `0x0` is set in `EAX` for `bInheritHandle`. The final parameter must be set with a MOV dereference, providing the PID, due to a lack of other suitable registers. PUSHAD is used to push these values onto the stack; the runtime address of `OpenProcess` is

reached, with its return address and parameters aligned. The handle to the target process is returned in EAX, allowing it to be saved in our cache for future use in other WinAPIs. If OpenProcess succeeds, a handle is returned in EAX; if it fails, it returns NULL. In this work, all OpenProcess patterns successfully yielded a handle, validating that each may be used in a modular fashion.

### F. OPENPROCESSTOKEN

It is used to create a handle for the access token of a specific process. The access token provides the security context of a process, including user credentials and privileges. This WinAPI is used as part of the privilege escalation phase in the process injection workflow. The injecting process can use the token handle for the specified process later as a required parameter for AdjustTokenPrivileges, allowing its privileges to be elevated. This allows for future actions in the process injection workflow to proceed without failing.

The ROP implementation for this WinAPI is straightforward; it has only one pattern, and it is set with MOV dereference, as shown in Table 6. The ProcessHandle parameter is set to −1 or 0xFFFFFFFF, indicating the current process. The DesiredAccess parameter is set to 0x20 or TOKEN_ADJUST_PRIVILEGES, which is required to disable or enable an access token's privileges. Finally, the TokenHandle argument is an out parameter, meaning that the output will be received there, so any pointer to valid memory is suitable. Once the WinAPI completes, this pointer receives a handle for the new access token for the specified process. If successful, OpenProcessToken places a valid token handle at the TokenHandle pointer and returns a nonzero value in EAX. Our testing confirmed this approach worked reliably as intended with the above values for parameters, setting the stage for privilege escalation.

### G. ADJUSTTOKENPRIVILEGES

AdjustTokenPrivileges disables or enables privileges in the access token for a target process, taking as input the token handle and a list of privileges to be adjusted. For process injection or other malicious functionality, it most commonly sets SeDebugPrivilege, which permits it to debug or manipulate external processes—a capability that otherwise would not possess. Without this step, later WinAPIs used for process injection would be blocked by security restrictions.

In process injection, AdjustTokenPrivileges completes the privilege escalation phase. With newly elevated privileges, later WinAPIs like CreateFileA, NtCreateSection, or NtMapViewOfSection, among others, can now access resources that were previously restricted, preventing the process injection from succeeding.

The ROP implementation for AdjustTokenPrivileges uses MOV dereference (see Table 7) because it has six parameters, which is too many for the PUSHAD technique. The needed values are loaded or generated via ROP and then written onto the stack. The TokenHandle parameter is provided by OpenProcessToken. As shown in Table 7, the DisableAllPrivileges argument is set to NULL, allowing it to adjust privileges based on the privileges specified in the NewState parameter. The NewState parameter is set to a pointer to the TOKEN_PRIVILEGES structure. Within this structure, PrivilegeCount is set to 0x1, indicating a single privilege being adjusted; it also contains another structure, LUID_AND_ATTRIBUTES. Each of these has a locally unique identifier (LUID) and its attributes are provided as a field. The LUID has a LowPart value of 0x20, for SE_DEBUG_PRIVILEGE, and the HighPart value is set to NULL. The Attributes field is set to 0x2, enabling SE_PRIVILEGE_ENABLED. All of the above structures and their respective values must be created in memory via ROP, using MOV dereference gadgets. The next set of parameters, BufferLength, PreviousState, and ReturnLength, are all set to NULL, as they are optional. If successful, AdjustTokenPrivileges returns a nonzero value, indicating that privileges were changed according to what had been specified in the LUID_AND_ATTRIBUTES structure. This allows the process injection to proceed without actions by later WinAPIs being denied access. We validated this approach by repeatedly testing it in the modern Windows platforms specified.

### H. CREATEFILEA

The CreateFileA function creates or opens a file, returning a handle so that the file can be interacted with, based on the specified attributes and flags. Process injection can take many forms, but in our approach, we wish to map Urlmon.dll into an external process and then inject our payload into it. This method is stealthier than directly allocating memory in the target process. Thus, we must obtain a handle to the Urlmon.dll, created using CreateFileA. That handle is then used by NtCreateSection, to create a section backed by this DLL, which gets mapped into the target. Afterward, its memory permissions are changed, allowing code injection and execution.

The ROP implementation of CreateFileA is set using MOV dereferences (see Table 8 for pattern). Because CreateFileA has seven parameters, it exceeds what the PUSHAD technique can handle. The lpFileName argument is set to the string \\??\\c:\\Windows\\SysWOW64\\urlmon.dll, the universal location for the 32-bit Urlmon.dll on modern Windows. However, other DLLs could be used and eventually mapped into the target process. The dwDesiredAccess parameter is set to 0x00120089 or GENERIC_READ, giving read access to the file. The dwShareMode parameter is set to 0x00000001 or FILE_SHARE_READ, allowing the file to be read by subsequent operations without the need to request access. The lpSecurityAttributes parameter is set to NULL, indicating that the handle generated by CreateFileA cannot be inherited by later child processes. While not relevant for process injection, this also avoids needing to create a structure via ROP. The

**TABLE 5.** Different patterns are depicted for the OpenProcess WinAPI.

| Reg. | Pattern 1 | Pattern 2 | Pattern 3 | Pattern 4 | Pattern 5 |
|---|---|---|---|---|---|
| EDI | RET 8 | RET 8 | Pop | Pop | RET 8 |
| ESI | RET | RET | OpenProcess | OpenProcess PTR | RET |
| EBP | RET | OpenProcess | Pop | Pop | OpenProcess PTR |
| ESP | Skip | Skip | Skip | Skip | Skip |
| EBX | OpenProcess | Jmp [EBP] | Jmp [EBP] | Jmp [EBP] | Jmp [EBP] |
| EDX | Return Address | Return Address | Return Address | Return Address | Return Address |
| ECX | dwDesiredAccess | dwDesiredAccess | dwDesiredAccess | dwDesiredAccess | dwDesiredAccess |
| EAX | bInheritHandle | bInheritHandle | bInheritHandle | bInheritHandle | bInheritHandle |

**TABLE 6.** A pattern for OpenProcessToken using Mov dereference.

| Operation | Mov Dereference Value |
|---|---|
| OpenProcessToken | Supplied by GetProcAddress |
| Return Address | Pointer to function return location |
| ProcessHandle | 0xFFFFFFFF |
| DesiredAccess | 0x20 for TOKEN_ADJUST_PRIVILEGES |
| TokenHandle | Pointer to NULL |

**TABLE 7.** A pattern for AdjustTokenPrivileges using Mov dereference.

| Operation | Mov Dereference Value |
|---|---|
| AdjustTokenPrivileges | Supplied by GetProcAddress |
| Return Address | Pointer to function return location |
| TokenHandle | Token handle from OpenProcessToken |
| DisableAllPrivileges | FALSE |
| NewState | Pointer to TOKEN_PRIVILEGES struct |
| BufferLength | NULL |
| PreviousState | NULL |
| ReturnLength | Pointer to NULL |

`dwCreationDisposition` is set to `0x00000003` or `OPEN_EXISTING`, meaning it will open the file only if it already exists. The `dwFlagsAndAttributes` argument is set to `0x00000860`, combining different file attributes. Finally, the `hTemplateFile` argument is set to `NULL`. Once the parameter values are written to memory, a stack pivot can occur, causing the WinAPI to be invoked with the return address and parameters properly aligned.

If `CreateFileA` succeeds, a handle to `Urlmon.dll` is returned in `EAX`. This handle, as with others, can be saved in our cache in memory, to be retrieved as needed. In our tests, `CreateFileA` worked regularly with these parameters. We tried alternative settings for parameters, but they were unreliable for subsequent WinAPIs. It is possible, however, that other parameter combinations could also work.

### *I. NTCREATESECTION*

`NtCreateSection` is a native API that creates a section object, which is shareable between processes, allowing a portion of virtual memory to be shared between different processes. Once a section is created, a view of it can be mapped into one or more processes, allowing for possible memory sharing between processes. In our injection workflow, `NtCreateSection` creates a section for `Urlmon.dll` to be mapped into the target process. Once mapped, it lets us write the payload into the target's virtual address space. Subsequent WinAPIs can then write the payload into this

**TABLE 8.** A pattern for CreateFileA using Mov dereference.

| Operation | Mov Dereference Value |
|---|---|
| CreateFileA | Supplied by GetProcAddress |
| Return Address | Pointer to function return location |
| lpFileName | String to DLL location |
| dwDesiredAccess | 0x120089 for GENERIC_READ |
| dwShareMode | 0x00000001 for FILE_SHARE_READ |
| lpSecurityAttributes | NULL |
| dwCreationDisposition | 0x3 for OPEN_EXISTING |
| dwFlagsAndAttributes | 0x860 |
| hTemplateFile | NULL |

memory, masquerading as a normal `Urlmon.dll`, and execute it remotely—avoiding a less stealthy `VirtualAlloc` call.

The ROP implementation for `NtCreateSection` uses the MOV dereference technique because its seven parameters exceed what PUSHAD can handle (see Table 9 for pattern). `SectionHandle` is initialized as a pointer to `NULL`; this is an out parameter that will be updated with a handle to the section. The `DesiredAccess` parameter is set to `0x10000000` or `GENERIC_ALL`, granting all possible rights to the section. The `ObjectAttributes` and `MaximumSize` parameters are set to `NULL`, as they are optional. The `SectionPageProtection` parameter is set to `0x00000002` or `PAGE_READONLY`, making the section initially read-only. The `AllocationAttributes` parameter is assigned the value `0x01000000` or `SEC_IMAGE`, indicating that the file, `Urlmon.dll`, is an executable image. Finally, the `FileHandle` is set to the handle obtained from the previous `CreateFileA` call, backing the section with `Urlmon.dll`.

If `NtCreateSection` succeeds, it returns a valid handle to the newly created section, placing it in the pointer indicated by `SectionHandle`. Additionally, the native API will return a `STATUS_SUCCESS` or `0x0` in `EAX`. We validated this API in WinDbg by checking `STATUS_SUCCESS` and ensuring the handle was written to the correct pointer. We also used Process Hacker to verify the handle indeed references a section. Validation testing has indicated that `NtCreateSection` is fully reliable as part of this process injection workflow.

**TABLE 9.** A pattern for NtCreateSection using Mov dereference.

| Operation | Mov Dereference Value |
|---|---|
| NtCreateSection | Supplied by GetProcAddress |
| Return Address | Pointer to function return location |
| SectionHandle | Pointer to NULL |
| DesiredAccess | NULL |
| ObjectAttributes | NULL |
| MaximumSize | NULL |
| SectionPageProtection | 0x00000002 for PAGE_READONLY |
| AllocationAttributes | 0x01000000 for SEC_IMAGE |
| FileHandle | This is handle from CreateFileA |

**TABLE 10.** A pattern for NtMapViewOfSection using Mov dereference.

| Operation | Mov Dereference Value |
|---|---|
| NtMapViewOfSection | Supplied by GetProcAddress |
| Return Address | Pointer to function return location |
| SectionHandle | SectionHandle from NtCreateSection |
| ProcessHandle | Process handle from OpenProcess |
| BaseAddress | Pointer to NULL |
| ZeroBits | NULL |
| CommitSize | NULL |
| SectionOffset | NULL |
| ViewSize | Pointer to the size of mapped block |
| InheritDisposition | 0x1 for VIEW_SHARE |
| AllocationType | NULL |
| Protect | 0x2 for PAGE_READONLY |

### J. NTMAPVIEWOFSECTION

`NtMapViewOfSection` is a native API that maps a view of a section into the virtual address space of the target process, using the specified memory protection and allocation type. This can allow a file, such as a DLL, to be mapped into the memory of a remote process by providing a section handle created from the DLL. In process injection, `NtMapViewOfSection` maps `Urlmon.dll` into the target process, creating an area where the payload can be injected, hidden in plain sight. We chose Urlmon.dll because it is a common system DLL. In our tests, mapping Urlmon.dll–and indeed all the actions in our case study–went undetected by Windows Defender, as shown in the Detection by Antivirus subsection.

The ROP implementation for this native API is complex, requiring ten parameters; as such, it is too large to use the PUSHAD approach and instead uses the MOV dereference technique (see Table 10 for pattern). Loading gadgets are used to set the parameter values prior to their being written to memory with a `MOV` dereference. The `SectionHandle` parameter is set to the handle created by `NtCreateSection`, which created a section backed by the `Urlmon.dll` file. The `ProcessHandle` parameter specifies the process that the view should be mapped to, and we use the handle obtained from `OpenProcess`. We provide a pointer to `NULL` as the `BaseAddress` parameter. This pointer will be updated with the `BaseAddress` of `Urlmon.dll` if it is successfully mapped into memory. The `ZeroBits`, `CommitSize`, and `SectionOffset` parameters are all set to `NULL`, as they are optional. The `ViewSize` parameter is set as a pointer to a `SIZE_T` variable; this is set to zero, allowing a complete view of the section to be mapped into memory. The `InheritDisposition` parameter is assigned the value `0x00000001` or `ViewShare`, allowing the view to be mapped to any possible child processes. The `AllocationType` parameter is set to `NULL`, meaning it is mapped with default protections from the section's file. Lastly, the `Win32Protect` parameter is set to `0x00000002` or `PAGE_READONLY`, matching the initial permissions used when the section was created. Once these parameters are in memory, a small pivot invokes `NtMapViewOfSection` with the return address and parameters properly aligned.

If `NtMapViewOfSection` succeeds, the `BaseAddress` pointer receives the base address of the mapped section, and it will return a `STATUS_SUCCESS` or `0x0` in `EAX`. Validation testing confirmed this API's reliability in our process injection workflow across multiple environments. It consistently mapped the section, backed by `Urlmon.dll`, into the target process.

### K. VIRTUALPROTECTEX

`VirtualProtectEx` modifies the memory protection of a region in the target process. This makes the memory executable, readable, or writable. This WinAPI allows the target process to be specified by the handle of the process. We use `VirtualProtectEx` to modify the memory protection of the newly mapped `Urlmon.dll`, giving it `PAGE_EXECUTE_READWRITE`. This step allows us to write the payload into `Urlmon.dll` and execute it remotely.

The ROP implementation of `VirtualProtectEx` uses MOV dereference (see Table 11 for pattern) because it has too many parameters for PUSHAD. The `hProcess` is set to the handle of our target process, provided earlier by `OpenProcess`. The `lpAddress` parameter is set to the address provided by `NtMapViewOfSection`, where the section was mapped into the target process's address space. The `dwSize` parameter is set to `0x00000201`, providing the size of the memory region to be changed, and the `flNewProtect` parameter is set to `0x00000040` or `PAGE_EXECUTE_READWRITE`, allowing our payload to be written to `Urlmon.dll` and executed. Finally, the `lpflOldProtect` parameter is set to a pointer that will receive the old access protection value. If successful, `VirtualProtectEx` returns a nonzero value, and the specified region's memory protection is changed. Tests using WinDbg confirmed that this WinAPI reliably supports our process injection workflow.

### L. WRITEPROCESSMEMORY

`WriteProcessMemory` copies data into a target's process memory, provided the specified area is available. This WinAPI, thus, serves as a way to transfer memory to external processes, with the appropriate handle. In our process injection workflow, `WriteProcessMemory` injects the

**TABLE 11.** A pattern for VirtualProtectEx using Mov dereference.

| Operation | Mov Dereference Value |
|---|---|
| VirtualProtectEx | Supplied by GetProcAddress |
| Return Address | Pointer to function return location |
| hProcess | Handle from OpenProcess |
| lpAddress | From Base Address of NtMapViewOfSection |
| dwSize | 0x201 |
| flNewProtect | PAGE_EXECUTE_READWRITE |
| lpflOldProtect | Output |

payload into the mapped `Urlmon.dll`, making the injected code available for remote execution.

In our approach, `WriteProcessMemory` is implemented via a combination of the MOV dereference and PUSHAD techniques. This is due it having five parameters; as such, it has only one pattern. As this pattern combines two approaches, it is set up so that when PUSHAD is executed, all parameters and the return address will be on the stack. Thus, the final three parameters are set up first using MOV dereferences. The third parameter, `lpBuffer`, is set to the address that contains the code that will be injected into `Urlmon.dll`, and the `nSize` parameter is set to `0x100`. Finally, `lpNumberOfBytesWritten` is set to a pointer to a variable that receives the number of bytes moved to the target process. Next, the PUSHAD technique is used to load the first two parameters and the return address into their designated registers. `ECX` is set to`hProcess`, using the handle that was provided by `OpenProcess`, as retrieved from our cache. Next, `EAX` is set to the `lpBaseAddress`; this value is `0x3000` bytes from the start of the base of `Urlmon.dll` that was mapped into the target process by `NtMapViewOfSection`. The runtime address of `WriteProcessMemory` is set to `EBX`, and the return address is set to `EDX`. Both `EDI` and `ESI` are set to a ROP nop, and `EBP` holds a pop gadget to skip the stack address from `ESP`. Upon executing PUSHAD, eventually the `WriteProcessMemory` API is reached, with the parameters and return address correctly set up. If `WriteProcessMemory` succeeds, the buffer is copied to the specified memory location, and `EAX` returns a nonzero value. Our validation testing confirmed the reliability of this WinAPI in the process injection workflow, using WinDbg to check both the value in `EAX` and to confirm that the payload had been written to the target process.

### M. CREATEREMOTETHREAD

`CreateRemoteThread` is the final WinAPI in our process injection workflow, creating a new thread in the virtual address space of the target process to launch the injected payload. This function takes seven parameters. `CreateRemoteThread` triggers execution of the injected payload. Here, our payload is proof-of-concept shellcode, but it could just as well be a malicious DLL. This can cause an attacker's payload to run within the context of a target process. Migrating to another process—by injecting the payload in an external process—can also greatly enhance exploit stability and reduce detection, assuming a suitable target is selected.

The ROP implementation for this WinAPI has only one pattern, due to it having six parameters; it combines the PUSHAD technique and MOV dereferences (see Table 13 for pattern). The first two parameters, `hProcess` and `lpThreadAttributes`, are handled by the PUSHAD technique, while the remaining are completed with MOV dereferences. In the ROP chain, the third parameter, `dwStackSize`, is created first, with its value set to `NULL`, accepting the default stack size. Next, `lpStartAddress` is the address where we aim to start execution; this will be the same address where the payload was written in `WriteProcessMemory`. The `dwCreationFlags` is set to zero, meaning remote execution begins at once; `lpThreadId` is set to `NULL`, so no thread identifier is returned. Upon executing PUSHAD, two ROP nops and a pop that consumes the stack value from `ESP` lead to the runtime address of `CreateRemoteThread` being executed, with its return address and parameters aligned.

If `CreateRemoteThread` succeeds, a return value to the handle of the new thread is given in `EAX`, and the payload executes. Our validation tests confirmed that proof-of-concept shellcode ran as expected, and `EAX` returned the correct handle.

### N. LOADLIBRARYA AND GETPROCADDRESS

`LoadLibraryA` and `GetProcAddress` serve as helper functions in our process injection workflow, resolving runtime addresses for WinAPIs. `LoadLibraryA` loads a module, providing a handle to it, which also happens to be its runtime base address. This handle is required as an input for `GetProcAddress`, which can be used to resolve the runtime address of an API.

To call APIs, there must be a pointer in the binary, from which we can obtain its runtime address. Alternatively, the address can be resolved via `GetProcAddress`. In the context of ROP, there may already be pointers for many of the previously mentioned WinAPIs. However, if not, then they can be resolved via `GetProcAddress` and `LoadLibraryA`, which can be used repeatedly in an exploit chain. In our testing, we examined a worst-case scenario where such pointers did not exist, so these functions were repeatedly used to resolve runtime addresses for WinAPIs. While these WinAPIs are not strictly required by the process injection workflow, they may still be invoked multiple times.

We created nine patterns for `LoadLibraryA` using the PUSHAD technique (see Table 14 for pattern). We will consider one representative pattern: `EAX` and `ESI` are both set to a ROP nop, and `EDI` is set to `RET 8`. `EBX` is set to a `JMP DWORD PTR [EBP]` gadget, and `EBP` is a pointer to `LoadLibraryA`. `ECX` is set to the `lpLibFileName` parameter, which is a pointer to a string for the module to be loaded, such as `"kernel32.dll"`. Once PUSHAD executes, the ROP nop gadgets and the `RET 8` allow the stack address from `ESP` to be skipped, leading to `JMP DWORD`

**TABLE 12.** A pattern for WriteProcessMemory using PUSHAD is shown at left. Mov dereference is used to complete the pattern.

| Reg. | Pattern Type | Pattern Value |
|---|---|---|
| EDI | RET | ROP nop |
| ESI | RET | ROP nop |
| EBP | POP | Pops ROP nop into register |
| ESP | SKIP | No value supplied |
| EBX | WriteProcessMemory | Supplied by GetProcAddress |
| EDX | Return Address | Pointer to function return location |
| ECX | hProcess | Handle from OpenProcess |
| EAX | lpBaseAddress | Base Address from NtMapViewOfSection |

| Operation | Mov Dereference Value |
|---|---|
| lpBuffer | Point to shellcode |
| nSize | 0x100 |
| *lpNumberOfBytesWritten | Pointer to the NULL |

**TABLE 13.** A pattern for CreateRemoteThread using PUSHAD is depicted at left. Mov dereference is used to complete the pattern.

| Reg. | Pattern Type | Pattern Value |
|---|---|---|
| EDI | RET | ROP nop |
| ESI | RET | ROP nop |
| EBP | POP | Pops ROP nop into register |
| ESP | SKIP | No value supplied |
| EBX | CreateRemoteThread | Supplied by GetProcAddress |
| EDX | Return Address | Pointer to function return location |
| ECX | hProcess | Handle from OpenProcess |
| EAX | lpThreadAttributes | NULL |

| Operation | Mov Dereference Value |
|---|---|
| dwStackSize | NULL |
| lpStartAddress | From Base Address of NtMapViewOfSection |
| lpParameter | NULL |
| dwCreationFlags | NULL |
| lpThreadId | NULL |

`PTR [EBP]`, which jumps to `LoadLibraryA`, with the return address and parameters aligned.

The ROP implementation of `GetProcAddress` has eleven patterns, each using the PUSHAD technique (see Table 15 for pattern). We will consider one sample pattern here: the `hModule` parameter takes the handle of the DLL, which is returned by `LoadLibraryA`. As the handle also happens to be its runtime address, this can be used if known, without the need to call `LoadLibraryA`, and the `lpProcName` parameter is a pointer to the string for the function name to be loaded. The `hModule` is placed in `ECX`, while the `lpProcName` is put in `EAX`. `EDI` is set to `RET 8`, skipping eight bytes or two potential pointers, thereby allowing the chain to continue without interruption. `ESI` is set to a ROP nop, and `EBP` holds the pointer to `GetProcAddress`. Finally, `EBX` is set to a `JMP DWORD PTR [EBP]` gadget. Upon executing PUSHAD, the `RET 8` and ROP nop result in the `GetProcAddress` pointer being jumped to, with the return address and parameters aligned.

If `LoadLibraryA` succeeds, it returns a handle in `EAX`, and if `GetProcAddress` succeeds, it places the specified API's runtime address in `EAX`. In our validation testing, both functions were tested and confirmed using `WinDbg`. In addition, they were repeatedly used throughout the process injection workflow to resolve runtime addresses for nearly all the functions used.

### O. RTLCREATEUSERTHREAD

Instead of using `CreateRemoteThread`, RtlCreateUserThread can be used optionally with additional setup for greater stealth. It creates a new thread in the virtual memory of a target process. Though it is similar to `CreateRemoteThread` in taking a process handle, start address, and thread attributes, this native API can also initiate a 64-bit payload in a 64-bit process while coming from a 32-bit binary, which would not be possible with `CreateRemoteThread`, as the architectures of the injecting and target process would need to match. In process injection, `RtlCreateUserThread` executes the injected payload in the target process. Compared to `CreateRemoteThread`, it is less likely to be detected by EDR because it is used far less frequently.

The ROP implementation combines the PUSHAD and MOV dereference technique (see Table 16), using one pattern. The first two parameters rely on the PUSHAD. The `ProcessHandle` uses the handle to the target process obtained by `OpenProcess`; the `pSecurityDescriptor` is optional and is set to `NULL`. The ROP pattern begins by setting `EDI` and `ESI` to a ROP nop, and `EBP` is set to a pop gadget, so that it can consume the stack address from `ESP`. `EBX` holds the runtime address of `RtlCreateUserThread`, and the return address is placed in `EDX`. The remaining parameters are set using MOV dereferences. `CreateSuspended`, `StackZeroBits`, `MaximumStackSize`, and `StackCommit` are optional and are set to `NULL`. The `StartAddress` is set to point to the payload, as this is where execution will begin. Next, `StartParameter` is set to `NULL`. `ThreadHandle` is set to point to a variable that will receive the handle of the created thread, and `ClientID` is set to `NULL`. After executing PUSHAD, the ROP nop gadgets lead to `RtlCreateUserThread` with the return address and parameters aligned.

If `RtlCreateUserThread` succeeds, it returns a handle to the new thread in `ThreadHandle`, and execution begins. Our testing confirmed that this native API worked as part of the process injection workflow when used instead of `CreateRemoteThread`.

**TABLE 14.** Different patterns are shown for the LoadLibraryA WinAPI.

| Reg. | Pattern 1 | Pattern 2 | Pattern 3 | Pattern 4 | Pattern 5 | Pattern 6 | Pattern 7 | Pattern 8 | Pattern 9 |
|---|---|---|---|---|---|---|---|---|---|
| EAX | lpLibFileName | ROP nop | lpLibFileName | lpLibFileName | ROP nop | ROP nop | ROP nop | lpLibFileName | lpLibFileName |
| ECX | Return Address | lpLibFileName | Return Address | Return Address | lpLibFileName | lpLibFileName | lpLibFileName | Return Address | Return Address |
| ESP | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip |
| EDI | ROP nop | Pop | Pop | RET 8 | RET 8 | ADD ESP, 0xC | ADD ESP, 0xC | ADD ESP, 0xC | ADD ESP, 0xC |
| ESI | RET 8 | LoadLibraryA PTR | LoadLibraryA PTR | ROP nop | ROP nop | ROP nop | LoadLibraryA PTR | ROP nop | LoadLibraryA PTR |
| EBP | ROP nop | Pop | Pop | LoadLibraryA PTR | LoadLibraryA PTR | LoadLibraryA PTR | ROP nop | LoadLibraryA PTR | ROP nop |
| EBX | LoadLibraryA PTR | Jmp [ESI] | ROP nop | ROP nop | Jmp[EBP] | Jmp[EBP] | Jmp [ESI] | ROP nop | ROP nop |
| EDX | Jmp [EBX] | Return Address | Jmp [ESI] | Jmp [EBP] | Return Address | Return Address | Return Address | Jmp [EBP] | Jmp [ESI] |

**TABLE 15.** Different patterns are depicted for the GetProcAddress (GPA) WinAPI.

| Reg. | Pattern 1 | Pattern 2 | Pattern 3 | Pattern 4 | Pattern 5 | Pattern 6 | Pattern 7 | Pattern 8 | Pattern 9 | Pattern 10 | Pattern 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| EAX | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName | lpProcName |
| ECX | hModule | hModule | hModule | hModule | hModule | hModule | hModule | hModule | hModule | hModule | hModule |
| ESP | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip | Skip |
| EDI | RET 8 | Pop | Pop | ADD ESP, 0xC | ADD ESP, 0xC | ADD ESP, 4 | ADD ESP, 0xC | ADD ESP, 0xC | RET 4 | ADD ESP, 0xC | ADD ESP, 4 |
| ESI | ROP nop | GPA PTR | GPA PTR | ROP nop | GPA PTR | GPA PTR | ADD ESP, 4 | RET 4 | ADD ESP, 4 | Pop | GPA PTR |
| EBP | GPA PTR | Pop | ADD ESP, 4 | GPA PTR | ROP nop | ADD ESP, 4 | GPA PTR | GPA PTR | GPA PTR | GPA PTR | Pop |
| EBX | Jmp [EBP] | Jmp [ESI] | Jmp [ESI] | Jmp [EBP] | Jmp [ESI] | Jmp [ESI] | Jmp [EBP] | Jmp [EBP] | Jmp [EBP] | Jmp [EBP] | Jmp [ESI] |
| EDX | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address | Return Address |

**TABLE 16.** A pattern for RtlCreateUserThread using PUSHAD is depicted at left. Mov dereference is used to complete the pattern.

| Reg. | Pattern Type | Pattern Value | Operation | Mov Dereference Value |
|---|---|---|---|---|
| EDI | RET | ROP nop | CreateSuspended | NULL |
| ESI | RET | ROP nop | StackZeroBits | NULL |
| EBP | POP | Pops ROP nop into register | StackReserved | NULL |
| ESP | SKIP | No value supplied | StackCommit | NULL |
| EBX | RtlCreateUserThread | Supplied by GetProcAddress | StartAddress | Shellcode address |
| EDX | Return Address | Pointer to function return location | StartParameter | NULL |
| ECX | ProcessHandle | Process handle from OpenProcess | ThreadHandle | Some writable address |
| EAX | SecurityDescriptor | NULL | ClientID | NULL |

### *P. SUMMARY OF THE PROCESS INJECTION TECHNIQUE*

The process injection technique presented here involves numerous APIs as part of a complex workflow, combining the PUSHAD technique and MOV dereferences. Each approach sets up the stack with the return address and required parameters before invoking the API. The workflow begins by using `VirtualAlloc` to create a memory region with appropriate permissions, serving as a cache for various parameters or structures to be stored at. This region is also intended to be where a custom `EnumerateProcesses` function is built. The workflow continues with the process enumeration phase, using `CreateToolhelp32Snapshot` to obtain a snapshot of active processes, and `Process32First` retrieves information about the first process. A custom enumeration function is then built by writing bytes into memory, which can be invoked as a custom ROP gadget. It takes the runtime address of `Process32Next` as input, allowing it to be repeatedly called hundreds of times as needed. The custom gadget also performs string comparisons on the target application name. Once the name is found, the gadget parses the `PROCESSENTRY32` structure to get the PID of the target process, which is used by `OpenProcess` to obtain a handle. Then, privilege escalation ensures no subsequent APIs are blocked. `OpenProcessToken` is used to elevate the injecting process, where ROP is running in memory, generating an access token. Next, `AdjustTokenPrivileges` enables powerful privileges, such as `SeDebugPrivilege` and `SePrivilegeEnabled`, letting the injecting process debug and manipulate the target.

The next phase creates and maps `Urlmon.dll` into the target binary. While other options exist, this approach offers additional stealth. Thus, `CreateFileA` is used to obtain a handle to `Urlmon.dll` with `GENERIC_READ` rights. Then `NtCreateSection` creates a section with `PAGE_READONLY` rights, backed by `Urlmon.dll`, and returns a section handle. A view of that section is mapped into the target process via `NtMapViewOfSection`, adding `Urlmon.dll` to the target application and returning its base address in `BaseAddress`. `VirtualProtectEx` then uses the target application's handle to change permissions to `PAGE_EXECUTE_READWRITE`, allowing us to write the payload directly into `Urlmon.dll`. Initially creating the section with `PAGE_READONLY` was intended to avoid potential problems and ensure the approach was portable across multiple versions of Windows. Next, `WriteProcessMemory` was used to write the payload into `Urlmon.dll`, and `CreateRemoteThread` was used to execute the payload.

Once the patterns were developed with appropriate parameters and the process injection workflow proved to be reliable, any of our novel patterns could be swapped in or out. Our approach does not match any existing process injection technique. Although it does perform remote thread injection, it does so in a more sophisticated, stealthy manner, using different APIs [8]. Similarly, it departs from all process injection methods by being implemented entirely by ROP. As such, it can be viewed as an original extension of remote

thread injection. Our goal is not to develop a new injection technique but to use this as a case study for advanced ROP development.

### Q. CLASSIC PROCESS INJECTION TECHNIQUE VIA ROP

While there are many process injection techniques, it was felt it was best to showcase an advanced technique. We also deployed a selection of the same APIs in a more classic approach to process injection, providing further validation of our patterns and demonstrating that they can be applied to other advanced exploit chains that use many APIs with complex handling of structures and needing to cache and reuse return values.

The exploit chain begins with `CreateToolhelp32Snapshot`, generating a snapshot of all active processes. `VirtualAlloc` then creates a memory region for caching handles and storing the `PROCESSENTRY32` structure and the custom enumeration function. Using this snapshot, `Process32First` retrieves details on the first process. A custom enumeration function is created by copying bytes into the newly allocated memory; it then calls `Process32Next` to iterate through the process list and identify the target process by name. Once found, its PID is extracted and used by `OpenProcess` to obtain a handle. `VirtualAllocEx` creates another allocation in the target process's address space, returning the base address in `EAX`. `WriteProcessMemory` writes the payload to the newly allocated memory, and `CreateRemoteThread` executes it. We also implemented a version of this exploit chain that replaced `CreateRemoteThread` with `RtlCreateUserThread`.

Validation testing of this classic injection technique showed that our novel patterns could be substituted for others. The APIs used parameters and ROP implementations similar to those in our main case study.

## V. RESULTS

The primary aim of this work had multiple goals: a) to demonstrate the soundness and resilience of our set of novel ROP patterns; b) to provide a methodology enabling advanced ROP in Windows, reaching a level of complexity not previously demonstrated; c) to create a custom `EnumerateProcess` function that resolves practical problems of implementing advanced functionality via pure ROP. We aimed to advance practical ROP capabilities on Windows. Success, in this context, means being able to invoke many Windows or native APIs solely through ROP, chaining them for sophisticated, real-world functionality. Because process injection is a popular exploitation method, we used it as a case study, noting that many of the involved APIs require complex setup—such as building Windows structures and requiring multiple handles, which can only be obtained by using multiple other WinAPIs, which may also require complex setup. In addition, each API produces a return value and sometimes additional output via out parameters, which must be saved in a cache for later use. With such an advanced exploit chain, all parameters and return values must be handled correctly for the process injection to succeed.

Reliability and accuracy of API calls, across different patterns, serve as our primary success metrics. The methodology must work consistently across various Windows builds, ensuring true portability, rather than merely developing an exploit that is tied to a single binary and not necessarily replicable on dissimilar binaries. Because many APIs require handles generated by previous calls, they cannot be tested in isolation; they must be evaluated within the relevant context of the greater exploit chain. We also evaluate the custom `EnumerateProcesses` function to verify functionality. It must iterate through active processes, compare executable names to the target process name, and locate the target to retrieve its PID. Failure at this stage would derail the entire process injection workflow. This evaluation also considers the modularity and extensibility of the ROP patterns developed for each API. The ability to adapt these patterns to different binaries and environments is an indicator of the methodology's practicality and effectiveness, demonstrating that the patterns work.

### A. FUNCTIONALITY ACROSS WINDOWS ENVIRONMENTS

To evaluate our methodology across various Windows environments, we divided it into different phases, each involving multiple APIs. Based on the success conditions defined in the implementation sections, we used WinDbg to verify each phase's completion on different Windows builds, reporting the results as percentages as shown in Table 17. These phases were tested on three Windows 10 builds and one Windows 11 build; older operating systems were considered to be not relevant or of interest due to their age.

The first phase we evaluated was Process Enumeration, involving `VirtualAlloc`, `CreateToolHelp32Snapshot`, `Process32First`, `Process32Next`, and `OpenProcess`. The second phase, Privilege Escalation, used `OpenProcessToken` and `AdjustTokenPrivileges`. Next, the Creating and Mapping File phase involved obtaining a handle to `Urlmon.dll` via `CreateFileA` and mapping it into the target application, e.g., VUPlayer or Discord, both 32-bit. The targets do not require any vulnerabilities. The Creating and Mapping File phase included `CreateFileA`, `NtCreateSection`, and `NtMapViewOfSection`, while the Payload Injection phase used `VirtualProtectEx` and `WriteProcessMemory`. Finally, the Payload Execution phase used `CreateRemoteThread`.

The results show that each phase succeeded across all tested Windows builds, demonstrating the resilience of our approach in using ROP to perform advanced, malicious functionality.

### B. PERFORMANCE AND RESOURCE UTILIZATION

The performance of our process injection technique was evaluated by measuring execution time, resource usage, and

**TABLE 17.** Functionality success rates across Windows environments.

| Workflow Stage | Win10 2004 | Win10 21H1 | Win10 22H2 | Win11 22H2 |
|---|---|---|---|---|
| Process Enumeration | 100% | 100% | 100% | 100% |
| Privilege Escalation | 100% | 100% | 100% | 100% |
| Creating & Mapping File | 100% | 100% | 100% | 100% |
| Payload Injection | 100% | 100% | 100% | 100% |
| Payload Execution | 100% | 100% | 100% | 100% |

**TABLE 18.** Flexibility and reusability of ROP patterns.

| Workflow Stage | Win10 2004 | Win10 21H1 | Win10 22H2 | Win11 22H2 |
|---|---|---|---|---|
| Process Enumeration | 3.5 s | 4.4 s | 3.8 s | 3.2 s |
| Privilege Escalation | 0.2 s | 0.2 s | 0.2 s | 0.3 s |
| Creating & Mapping File | 0.3 s | 0.28 s | 0.25 s | 0.35 s |
| Payload Injection | 0.1 s | 0.09 s | 0.08 s | 0.12 s |
| Payload Execution | 0.15 s | 0.12 s | 0.1 s | 0.17 s |

adaptability under various scenarios. This assessment focused on confirming that the workflow remains efficient, even for complex chains involving multiple APIs. We measured the execution time of our ROP process injection technique on Windows 10 Build 2004, Windows 10 Build 21H1, Windows 10 Build 22H2, and Windows 11 Build 22H2. The workflow was divided into phases mirroring those in the previous section. Overall, execution times were relatively fast, with minor variations.

Process enumeration ranged from 3.2 to 4.4 seconds, privilege escalation from 0.2 to 0.3 seconds, file creation and mapping from 0.25 to 0.35 seconds, payload injection from 0.08 to 0.12 seconds, and payload execution from 0.1 to 0.17 seconds, as seen in Table 18. Process enumeration took the longest, likely due to its more complex actions and repeated API calls. These time differences may stem from OS build variations or other environmental factors. In a real-world implementation of ROP, such a task would be done only once—not repeatedly—and these would be considered acceptable times.

### C. FLEXIBILITY AND REUSABILITY OF ROP PATTERNS

The modularity and reusability of our novel ROP patterns significantly enhance the practical application of ROP techniques. Previously, largest number of documented patterns for a single API was two, for `VirtualAlloc` and `VirtualProtect`. These patterns greatly simplify ROP usage, as they can make complex actions—such as establishing a return address, setting needed parameters, and invoking an API—much simpler. In addition, the PUSHAD techniques uses far fewer gadgets than would be required if using MOV dereference.

These patterns were created through extensive, time-consuming experimentation and trial-and-error. Each pattern was tested across three Windows 10 builds and one Windows 11 build, with each API being debugged in WinDbg to verify success according to criteria established in its implementation section. The success rate for all patterns was 100%, as shown in Table 19, demonstrating their robustness and reliability.

The modularity of these patterns allows them to be swapped, greatly expanding the potential attack surface. Creating multiple patterns for each API, when possible, solves the problem of limited gadget availability, which could otherwise render an attack infeasible. For example, loading a required value into a specific register may be impossible in one pattern, but another pattern could place the same value into a different register, one for which there were many gadgets. This methodology, thus, redefines the attack surface. With a larger set of workable patterns, the overall portability of this approach is significantly enhanced, making it far more likely that the process injection could be deployed on a given binary.

**TABLE 19.** Evaluation of ROP patterns for each WinAPI.

| WinAPI | Num. of Patterns | Success Rate |
|---|---|---|
| LoadLibraryA | 9 | 100% |
| GetProcAddress | 11 | 100% |
| CreateToolhelp32Snapshot | 11 | 100% |
| VirtualAlloc | 2 | 100% |
| Process32First | 10 | 100% |
| Process32Next | 10 | 100% |
| OpenProcess | 5 | 100% |
| OpenProcessToken | 1 | 100% |
| AdjustTokenPrivileges | 1 | 100% |
| CreateFileA | 1 | 100% |
| NtCreateSection | 1 | 100% |
| NtMapViewOfSection | 1 | 100% |
| VirtualProtectEx | 1 | 100% |
| WriteProcessMemory | 1 | 100% |
| CreateRemoteThread | 1 | 100% |

### D. DETECTION BY ANTIVIRUS AND EDR

A primary reason for using a pure ROP-only approach to malicious functionality is its high resilience to detection. To evaluate whether security solutions would flag our ROP payload, we conducted both static scans (e.g., VirusTotal, HybridAnalysis) and real-time trials against multiple EDR tools. Below, we provide details on these experiments and offer an explanation of why our ROP-only approach minimizes detection.

We tested four process injection techniques against antivirus and EDR solutions. *Variation I*, the primary technique examined in this work, maps a section into the target process using `NtCreateSection` and `NtMapViewOfSection` and then calls `CreateRemoteThread` to initiate execution. In all four variations, multiple calls to `LoadLibraryA` and `GetProcAddress` are used as needed to dynamically resolve API addresses. *Variation II* is identical to Variation I except that it uses `RtlCreateUserThread` instead of `CreateRemoteThread`, which can sometimes be more stealthy. *Variation III* follows a more classic approach, with process enumeration and privilege escalation the same as before. Instead of creating a section from `urlmon.dll`, `VirtualAllocEx` instead allocates memory directly into the target process. `WriteProcessMemory` loads the payload, and `CreateRemoteThread` starts execution. *Variation IV* is identical to Variation III, except that it uses `RtlCreateUserThread` for thread creation within the

target process.

We first tested the payloads from our four distinct ROP-based process-injection variants by uploading the resulting binary files to VirusTotal, which incorporates 61 antivirus engines, and to HybridAnalysis. The payload consisted of ROP gadgets and assorted filler (for POP gadgets), in binary format. We tested the payloads from our four process injection variations by uploading them to VirusTotal, to see if any of the 61 antivirius engines used by VirusTotal would flag it as malicious. In all cases, with VirusTotal and HybridAnalysis, as shown in Table 20, there was no static detection that the payload was malicious, i.e. that it was comprised of ROP gadgets that could invoke complex malicious functionality. Similar functionality implemented in an executable, e.g. a malware sample, would often be flagged as malicious. Given that ROP-only payloads are rarely flagged by signature-based or static, heuristic detection, we did not anticipate any positive results.

We next tested all four variants on Windows 10, configuring each with default or recommended settings for three popular EDR solutions, including Windows Defender, Wazuh EDR, and Elastic EDR. Each process-injection variant was executed 20 times on a fully patched system. When repeatedly executing each of the ROP-only process injection attacks and verifying their success via the debugger, none of the tested EDR solutions was able to detect our ROP process injection variants; we monitored EDR dashboards and logs for any alerts, warnings, or other possible indicators. For each process injection variant, on all EDR solutions, no security events were triggered, and the ROP process injection variants were successfully executed without issue (see Table 21).

In addition, we observe that one of our ROP chains is a classic variation of process injection, while the other is completely novel, with no previous analogs existing in the wild. Yet neither was detected. If the classic approach had been implemented directly in a higher-level language, it almost certainly would have been flagged by most EDRs. Thus, we can conclude that implementing process injection via ROP does confer a special benefit of minimizing detection. These successful outcomes support our hypothesis that ROP-only process injection effectively evades EDR heuristics. Unlike shellcode approaches, ROP reuses legitimate instructions from known modules from the executable, and it avoids typical indicators of process injection, such as newly suspicious PE sections or recognizable shellcode patterns. Based on these results, we conclude that ROP-only process injection greatly reduces the chances of detection in typical EDR environments.

**TABLE 22.** The `EnumerateProcesses` function is effective at performing string comparisons to identify the target process and retrieve its PID.

| Variation | EnumerateProcess Success |
|---|---|
| Process Inject. I | 100% |
| Process Inject. II | 100% |
| Process Inject. III | 100% |
| Process Inject. IV | 100% |

**TABLE 20.** VirusTotal and Hybrid Analysis could not statically detect the ROP payloads of the four process injection variations as malicious.

| Variation | VirusTotal | Hybrid Analysis |
|---|---|---|
| Process Inject. I | 0% | 0% |
| Process Inject. II | 0% | 0% |
| Process Inject. III | 0% | 0% |
| Process Inject. IV | 0% | 0% |

**TABLE 21.** Windows Defender, Wazuh EDR, and Elastic EDR did not detect the four ROP-only process injection variations as malicious.

| Variation | Win. Defender | Wazuh EDR | Elastic EDR |
|---|---|---|---|
| Process Inject. I | 0% | 0% | 0% |
| Process Inject. II | 0% | 0% | 0% |
| Process Inject. III | 0% | 0% | 0% |
| Process Inject. IV | 0% | 0% | 0% |

### E. TESTING AND VALIDATION OF THE ENUMERATEPROCESSES FUNCTION

One of our chief contributions was the design of the `EnumerateProcesses` function, which resolved the long-standing issues of performing iterative string comparisons via ROP. Rather than relying on a collection of string-comparison gadgets that may or may not exist in a given binary, `EnumerateProcesses` writes a custom function to memory. It invokes `Process32Next`, compares the retrieved process name using `REPE CMPSB`, looping until either a match is found or there are no more processes. When a match is found, the function extracts the process identifier (PID) from the `PROCESSENTRY32` structure, storing it at a known location for subsequent ROP calls. We conducted a total of 600 enumeration attempts, as seen in Table 22. `EnumerateProcesses` achieved a 100% success rate in returning the correct PID of the target process; no alerts were triggered. Note that if the target process is not active, then it cannot be located.

## VI. DISCUSSION

A ROP-only approach offers significant benefits in evading EDR as shown in Table 21 compared to the far more traditional ROP-to-shellcode approach. This can create a blind spot for detection tools, as ROP enables advanced malicious functionality that may be overlooked. Furthermore, because no new executable memory allocations are introduced, ROP-only attacks evade the EDR alerts typically triggered by suspicious RWX regions. Although our case study did employ `VirtualAlloc`, we could have avoided allocating executable memory. This approach also reduces call-stack anomalies, since API calls originate from trusted modules rather than external addresses. All instructions are borrowed from existing module gadgets, so signature-based tools are less likely to flag them. The payload consists only of return addresses and data, thwarting static detection [43], [44].

The ROP-only approach, stemming from trusted modules, helps reduce the shellcode footprint, as we avoid having a classic shellcode buffer that might be detected due to signature detection and heuristics [45], [46]. A ROP chain

resides in non-executable memory and is not recognizable as code; it is simply data in the form of gadget addresses referencing legitimate modules. This reduces the chances of detection during memory scans. Many EDR solutions detect shellcode by scanning for familiar opcode patterns or suspicious instruction sequences. For instance, tools like SHAREM [47] can readily detect shellcode-like behavior, such as accessing the PEB via `fs:[0x30]` or `gs:[0x60]`, whereas our ROP-only approach avoids recognizable shellcode patterns. A ROP payload is also far less human-readable: while SHAREM can fully disassemble encoded shellcode samples, no tool provides equivalent analysis for ROP. There is no standardized way to disassemble ROP as one might do with shellcode using SHAREM, Ghidra, or IDA Pro [48]. A reverse engineer must painstakingly reconstruct which gadgets are invoked and in what sequence, typically by stepping through each gadget and noting the corresponding assembly. This greatly obscures the attack's logic because a labyrinth of return addresses must be manually decoded, given that there is no straightforward method to accomplish this, unlike running SHAREM on shellcode to identify all APIs and parameters.

Additionally, shellcode can be self-modifying, resolving API addresses at runtime and performing other dynamic actions; these often trigger heuristic detections. Process injection may also involve dropping or reflectively loading a DLL, which monitoring tools can block; our approach avoids the need to drop or load any DLL. In addition, some ROP approaches that merely bypass DEP to launch shellcode, which may then download a second-stage payload; this is a step that could be detected or blocked. In contrast, our approach minimizes such unnecessary network indicators of compromise. Our approach is also fileless, requiring no additional downloads (e.g., DLLs or temporary files), thereby keeping all malicious components in memory and further minimizing detection.

Beyond this, to evade user-mode API hooks, low-level Windows syscalls could be used instead of WinAPIs or native APIs. In our research, we chose native APIs for simplicity. However, with additional work to supply the syscall system number (SSN), these could be used to invoke Windows syscalls directly, evading most EDR solutions [49], [50]. Thread start monitors are largely bypassed because the instruction pointer keeps returning to recognized module addresses, and even at the final injection stage, the new thread begins execution in Urlmon.dll rather than newly allocated memory. All of these factors make a pure ROP chain—free of shellcode—significantly less detectable than more prevalent approaches that pivot to shellcode. Approaches with shellcode are often more readily detected, often allocate new executable memory or create suspicious return addresses outside known modules, making them easier to spot via memory scanning and API-hooking [43].

This research bridges the gap between theoretical capabilities and real-world Windows exploitation. With helper APIs such as `LoadLibrary` and `GetProcAddress`, this exploitation consists of 30 APIs, which is a tenfold increase on the most we had observed in the wild, demonstrating a much higher level of sophistication. These capabilities show that advanced malicious functionality can be achieved exclusively through ROP. Our method shows a previously undocumented form of process injection via ROP, requiring significant setup, but it remains adaptable across multiple binaries through our library of novel gadget patterns. Our approach also provides a demonstration of Turing-completeness for ROP. Turning-completeness in ROP is not a new concept, but most real-world ROP usage is minimal, and our approach demonstrates highly complex functionality, such as privilege escalation and process enumeration, done entirely via ROP, expanding the range of post-exploitation tasks that advanced attackers or red teams might script using only ROP.

The creation of a library of nearly 150 ROP patterns to provide a modular approach to ROP chain construction makes the approach to be adaptable across many binaries. Without such patterns, advanced ROP on Windows, particularly those involving multiple APIs, would be severely limited, as the inability to complete even one API chain would disrupt the entire exploit. A lack of documented patterns, coupled with the difficulty of creating them, may have been one of the barriers preventing ROP in Windows from advancing beyond its theoretical limitations. If crafting even one API pattern is time-intensive or hindered by insufficient gadgets, the entire process becomes prohibitively costly. In addition, a working pattern might only apply to one binary and not transfer easily to others. As additional APIs are added to an exploit chain, the likelihood of failure due to insufficient gadgets increases. Thus, the contribution of creating and documenting these novel patterns provides a reusable blueprint for constructing ROP chains that can be adaptable to numerous binaries. While our approach focuses on process injection, the focus could be shifted to other offensive security tasks, such as dumping credentials or modifying registry entries. Those tasks could be supported using or extending our parameter-loading library. Before our work, only a handful of such patterns [42] were informally documented, most notably two examples each for `VirtualProtect` and `VirtualAlloc`.

From an offensive security perspective, the greatest hurdle in ROP exploitation is often the scarcity of gadgets in certain binaries. We address this issue by introducing a library of interchangeable API patterns, documenting every known pattern variation required for process injection. This modularity redefines the attack surface, allowing exploitation even in binaries with limited gadgets, where ROP might otherwise be infeasible.

Historically, on Windows, complex ROP workflows invoking advanced malicious functionality with many APIs seem to have been largely theoretical. Our research demonstrates a fully operational approach, chaining multiple APIs via ROP within a single workflow. Thus, this work extends and broadens the practical application of ROP on Windows. Finally, the extensive documentation and validation of patterns fill a major gap in the academic literature. The lack

of resources for creating and testing ROP patterns arguably has hindered the progress of ROP as an offensive security technique, beyond merely bypassing mitigations, e.g., DEP, as a close examination of ROP exploits on Exploit-DB reveals. Our contributions provide a valuable resource for the security community, spurring further development of ROP techniques. This also encourages improved defensive security, to limit ROP's effectiveness.

## VII. CONCLUSIONS AND FUTURE DIRECTIONS

This work bridges the gap between theoretical possibilities and practical exploitation by demonstrating process injection techniques using only ROP on Windows. We focused on one case study, including a variant approach with `RtlCreateUserThread`, and provided a summary of a classic process injection technique, as well as a variant using `RtlCreateUserThread` [51]. This work is the first and only to implement process injection purely via ROP, at least when considering what is publicly available; we are not aware of any private research that does so either. A major contribution of this work is the development of an extensive library of ROP patterns, overcoming the constraints imposed by scarce gadget availability. We provide a blueprint for future research by documenting nearly 150 patterns and demonstrating their successful use in complex exploit chains. We also demonstrate the transformative potential of ROP in offensive security.

In a practical sense, unlike most code-reuse attacks that disable mitigation and pivot to shellcode, our approach achieves robust functionality solely via ROP. For process injection, no technique had been fully implemented via ROP prior to our efforts. Avoiding reliance on shellcode to implement much of the functionality is a major departure from the prevalent practice on Windows. Our pattern library for parameter loading is a strong offensive security contribution, far beyond the relatively few patterns previously documented.

The nature of some vulnerabilities will inherently limit payload size, and due to this, some advanced ROP chains may not be possible. However, in binaries lacking such constraints, our approach is fully viable. Finally, this work demonstrates the largely untapped potential of ROP when used with patterns. This work advances the state of the art for the practical application of ROP in security research by documenting reusable patterns and detailing the practical aspects of chaining together many APIs via ROP for advanced functionality. This work will be a foundation for advancing ROP techniques on Windows and beyond.

This paper provides many possibilities for future work. One promising direction is the full automation of ROP chain construction. With this set of patterns or others of a similar nature, partial or complete ROP chains could be created via automation, significantly reducing manual effort. A future tool could potentially take as input a target binary, capture the gadgets, and deploy combinations from any of the nearly 150 patterns to form a code-reuse attack for process injection. In addition, automation could be extended to pattern creation itself, using machine learning or AI instead of time-consuming manual experimentation. This could greatly shorten development and testing cycles for patterns.

The modularity of our methodology and the accompanying pattern library can open the door to more advanced functionality being carried out via ROP. After all, other complex tasks may involve using outputs from one API, e.g., a handle, as inputs for future operations, and may require building complicated Windows structures in memory. Arguably, one of the chief reasons why highly advanced ROP functionality is not widely implemented is the time-intensive pattern development process and the lack of documentation.

While our current work focuses on Windows, the concept of building sets of patterns or templates for loading parameters into an API and then invoking it can be readily adapted to other platforms like Linux or ARM. For instance, on Linux, one could identify gadgets and parameter-loading patterns for libc functions or system calls, much like our current approach focused on Windows. On ARM, these patterns would need to account for ARM-specific calling conventions. The concept is the same for all; building a reusable gadget library substantially lowers manual effort in advanced exploitation.

While this work advances offensive security, we hope that it leads the antivirus industry and academic community to focus more on the detection of ROP, as this would significantly improve defensive security. Novel approaches to analyzing potentially large ROP chains would significantly bolster defensive security. Our experiments confirm that improved detection of ROP-only attacks is sorely needed.

## REFERENCES

[1] Microsoft, "Data execution prevention," 2023. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/memory/data-execution-prevention
[2] N. Stojanovski, M. Gusev, D. Gligoroski, and S. J. Knapskog, "Bypassing data execution prevention on microsoftwindows xp sp2," in The Second International Conference on Availability, Reliability and Security (ARES'07). IEEE, 2007, pp. 1222–1226.
[3] B. Cooke, "Cloudme 1.11.2 - buffer overflow rop (dep,aslr)," https://www.exploit-db.com/exploits/48840, Sep. 2020, exploit Database ID: 48840. CVE: CVE-2018-6892. Platform: Windows. Local exploit using ROP to achieve Add-Admin functionality.
[4] K. Lu, D. Zou, W. Wen, and D. Gao, "derop: removing return-oriented programming from malware," in Proc. 27th Annual Computer Security Applications Conference, 2011, pp. 363–372.
[5] L. Davi, A.-R. Sadeghi, D. Lehmann, and F. Monrose, "Stitching the gadgets: On the ineffectiveness of coarse-grained control-flow integrity protection," in 23rd USENIX Security Symposium (USENIX Security 14), 2014, pp. 401–416.
[6] J. Starink, M. Huisman, A. Peter, and A. Continella, "Understanding and measuring inter-process code injection in windows malware," in International Conference on Security and Privacy in Communication Systems. Springer, 2023, pp. 490–514.
[7] A. Klein and I. Kotler, "Windows process injection in 2019," Black Hat USA, vol. 2019, 2019.
[8] MITRE Corporation, "Process injection," https://attack.mitre.org/techniques/T1055/, 2023, accessed: 2024-12-03.
[9] c0de90e7, "Ghostwriting injection technique," https://github.com/c0de90e7/GhostWriting, 2007, accessed: 2024-12-03.
[10] A. Peslyak, "Getting around non-executable stack (and fix)," Bugtraq mailing list archives, 1997.
[11] N. Nergal, "The advanced return-into-lib (c) exploits: Pax case study," Phrack, vol. 4, p. 58, 2001.

[12] T. Durden, "Bypassing pax aslr protection," Phrack magazine, vol. 59, no. 9, p. 9, 2002.
[13] H. Shacham, "The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)," in Proceedings of the 14th ACM conference on Computer and communications security, 2007, pp. 552–561.
[14] R. Roemer, E. Buchanan, H. Shacham, and S. Savage, "Return-oriented programming: Systems, languages, and applications," ACM Transactions on Information and System Security (TISSEC), vol. 15, no. 1, pp. 1–34, 2012.
[15] J. Salwan, "Ropgadget: A tool to find rop gadgets in binaries," https://github.com/JonathanSalwan/ROPgadget, 2011, accessed: 2024-12-03.
[16] P. V. Eeckhoutte, "Corelan repository for mona.py," https://github.com/corelan/mona, accessed: 2024-12-03.
[17] S. Schirra, "Ropper: A tool to search for rop gadgets in binaries," https://github.com/sashs/Ropper, accessed: 2024-12-03.
[18] B. Brizendine and S. S. Kusuma, "Rop_rocket," https://github.com/Bw3ll/ROP_ROCKET, 2023, accessed: 2024-12-03.
[19]
[20] T. Bletsch, X. Jiang, V. W. Freeh, and Z. Liang, "Jump-oriented programming: a new class of code-reuse attack," in Proceedings of the 6th ACM symposium on information, computer and communications security, 2011, pp. 30–40.
[21] S. Checkoway, L. Davi, A. Dmitrienko, A.-R. Sadeghi, H. Shacham, and M. Winandy, "Return-oriented programming without returns," in Proc. ACM conference on computer and communications security, 2010, pp. 559–572.
[22] B. Brizendine and A. Babcock, "Pre-built JOP chains with the JOP ROCKET: bypassing DEP without ROP," Black Hat Asia, 2021.
[23] ——, "A novel method for the automatic generation of JOP chain exploits," in National Cyber Summit (NCS) Research Track 2021. Springer, 2022, pp. 77–92.
[24] K. Z. Snow, F. Monrose, L. Davi, A. Dmitrienko, C. Liebchen, and A.-R. Sadeghi, "Just-in-time code reuse: On the effectiveness of fine-grained address space layout randomization," in Proc. IEEE symposium on security and privacy, 2013, pp. 574–588.
[25] S. Sayeed, H. Marco-Gisbert, I. Ripoll, and M. Birch, "Control-flow integrity: Attacks and protections," Applied Sciences, vol. 9, no. 20, p. 4229, 2019.
[26] National Security Agency, Cybersecurity Directorate, "Hardware control flow integrity for an it ecosystem," https://github.com/nsacyber/Control-Flow-Integrity/blob/master/paper/Hardware%20Control%20Flow%20Integrity%20for%20an%20IT%20Ecosystem.pdf, 2015, accessed: 2024-12-03.
[27] Intel Corporation, "A technical look at intelś control-flow enforcement technology," https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html, 2020, accessed: 2024-12-03.
[28] F. Schuster, T. Tendyck, C. Liebchen, L. Davi, A.-R. Sadeghi, and T. Holz, "Counterfeit object-oriented programming: On the difficulty of preventing code reuse attacks in c++ applications," in Proc. IEEE Symposium on Security and Privacy, 2015, pp. 745–762.
[29] N. Carlini and D. Wagner, "ROP is still dangerous: Breaking modern defenses," in 23rd USENIX Security Symposium, 2014, pp. 385–399.
[30] J. Ganz and S. Peisert, ASLR: How robust is the randomness? IEEE, 2017.
[31] L. Binosi, G. Barzasi, M. Carminati, S. Zanero, and M. Polino, "The illusion of randomness: An empirical analysis of address space layout randomization implementations," in Proc. ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 1360–1374.
[32] T. Barabosch, S. Eschweiler, and E. Gerhards-Padilla, "Bee master: Detecting host-based code injection attacks," in Detection of Intrusions and Malware, and Vulnerability Assessment: Proc. Int. Conf. Springer, 2014, pp. 235–254.
[33] K. Konrad, "Exploring advanced process injection techniques in windows malware," Master's thesis, EPFL, 2023.
[34] MITRE Corporation, "MITRE ATT&CK®," https://attack.mitre.org, 2024, accessed: 2024-12-03.
[35] M. Nasereddin and R. Al-Qassas, "A new approach for detecting process injection attacks using memory analysis," International Journal of Information Security, vol. 23, no. 3, pp. 2099–2121, 2024.
[36] J. R. Dora and L. Hluchy, "Process injection and migration techniques: * strategies to bypass security software of network communication after a successful reverse shell," in Proc. IEEE 23rd World Symposium on Applied Machine Intelligence and Informatics (SAMI), 2025, pp. 000 017–000 022.
[37] J. Wang, C. Ma, Z. Li, H. Yuan, and J. Wang, "Procguard: Process injection behaviours detection using fine-grained analysis of api call chain with deep learning," in Proc. IEEE Int. Conf. on Trust, Security and Privacy in Computing and Communications (TrustCom), 2022, pp. 778–785.
[38] T. M. Lewis and B. P. Rimal, "Effects of removing user-land hooks in endpoint protection during attack experiments," IEEE Access, vol. 12, pp. 15 820–15 844, 2024.
[39] O. Chechik, "Banking trojan techniques: How financially motivated malware became infrastructure," Unit 42, Palo Alto Networks blog, Oct 2022, accessed: April 16, 2025. [Online]. Available: https://unit42.paloaltonetworks.com/banking-trojan-techniques/
[40] A. Mazor, "Stackbombing – next gen process injection technique," Digital Whisper, no. 119, 2020, accessed: 2024-12-03.
[41] O. Yair, "Rop - from zero to nation state in 25 minutes," https://2020.bsidestlv.com/agenda/rop___from_zero_to_nation_state_in_25_minutes/, 2020, accessed: 2024-12-03.
[42] P. V. Eeckhoutte. (2010, Jun.) Exploit writing tutorial – part 10 – chaining dep with rop – the rubik's cube. Accessed: 2025-04-05. [Online]. Available: https://www.corelan.be/index.php/2010/06/16/exploit-writing-tutorial-part-10-chaining-dep-with-rop-the-rubikstm-cube/
[43] D. C. D'Elia, L. Invidia, and L. Querzoni, "Rope: Covert multi-process malware execution with return-oriented programming," in Computer Security–ESORICS 2021: 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part I 26. Springer, 2021, pp. 197–217.
[44] C. Ntantogian, G. Poulios, G. Karopoulos, and C. Xenakis, "Transforming malicious code to ROP gadgets for antivirus evasion," IET Information Forensics and Security, vol. 13, no. 6, pp. 570–578, 2019.
[45] F. Kasler. (2023) Deep sea phishing. Blog post. Accessed: 2025-04-05. [Online]. Available: https://posts.specterops.io/deep-sea-phishing-pt-1-092a0637e2fd
[46] M. Polychronakis, K. G. Anagnostakis, and E. P. Markatos, "Comprehensive shellcode detection using runtime heuristics," in Proceedings of the 26th Annual Computer Security Applications Conference, 2010, pp. 287–296.
[47] B. Brizendine, J. Hince, A. Babcock, T. Abdelmotaleb, S. Walker, and S. VandenHoek, "SHAREM: shellcode analysis framework with emulation, a disassembler, and timeless debugging," in Proceedings of the Virus Bulletin Conference, 2022.
[48] P. Borrello, E. Coppa, and D. C. D'Elia, "Hiding in the particles: When return-oriented programming meets program obfuscation," in 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). IEEE, 2021, pp. 555–568.
[49] O. Chechik and O. Ozer, "A deep dive into malicious direct syscall detection," https://www.paloaltonetworks.com/blog/security-operations/a-deep-dive-into-malicious-direct-syscall-detection/, 2024, feb. 13, 2024.
[50] M. Hutchins. (2023, Dec.) An introduction to bypassing user mode edr hooks. Accessed: 2025-04-05. [Online]. Available: https://malwaretech.com/2023/12/an-introduction-to-bypassing-user-mode-edr-hooks.html
[51] R. Mudge, "Cobalt strike's process injection: The details," 2019. [Online]. Available: https://www.cobaltstrike.com/blog/cobalt-strikes-process-injection-the-details-cobalt-strike

BRAMWELL BRIZENDINE is an assistant professor at the University of Alabama in Huntsville. He specializes in software exploitation, code-reuse attacks, reverse engineering, shellcode analysis, offensive security, and malware analysis. Dr. Brizendine presented his research at many top cybersecurity and hacking conferences. He serves on the review board for Black Hat Arsenal.

SHIVA SHASHANK KUSUMA is a graduate student at the University of Alabama in Huntsville, where he earned a Master's in Computer Science. His research focuses on code-reuse attacks, reverse engineering, and software exploitation.

BHASKAR P. RIMAL (SENIOR MEMBER, IEEE) is currently an Assistant Professor with the Department of Computer Science, University of Idaho, Moscow, USA. He was an Assistant Professor at Dakota State University, Madison, SD, USA. He was a Visiting Assistant Professor and an Academic Specialist with the Department of Computer Science, Tennessee Tech University, Cookeville, TN, USA. He was a Visiting Scholar with the Department of Computer Science, Carnegie Mellon University (CMU), Pittsburgh, PA, USA. He is currently the Associate Editor for the IEEE ACCESS and Associate Editor for the IEEE TRANSACTIONS ON TECHNOLOGY AND SOCIETY. He was the Technical Editor of the IEEE NETWORK, an Associate Technical Editor of the IEEE COMMUNICATIONS MAGAZINE, an Associate Editor of the EURASIP Journal on Wireless Communications and Networking, and an Editor of the Internet Technology Letters. Dr. Rimal is a Senior Member of ACM and a Senior Professional member of the ASEE.